

.NET PROGRAMMING



ASP.NET CORE MVC

**Complete Learning & Interview Preparation
Guide**

From Beginner to Advanced Concepts

DEPENDENCY INJECTION (DI)

What is Dependency Injection?

Dependency Injection (DI) is a design pattern where required objects (dependencies) are provided to a class from outside instead of creating them inside the class.

Simple Definition:

"Don't create objects yourself, let ASP.NET Core provide them."

Why Do We Need DI?

Without DI:

- Tight Coupling
- Difficult Testing
- Difficult Maintenance
- Difficult to Replace Implementations

With DI:

- Loose Coupling
 - Easy Unit Testing
 - Easy Maintenance
 - Better Scalability
 - Easy to Replace Services
-

Real Life Example

Without DI

```
public class Car
{
    private Engine _engine = new Engine();
}
```

Problem:

Car is tightly coupled with Engine.

With DI

```
public class Car
{
    private readonly Engine _engine;
    public Car(Engine engine)
```

```
{  
    _engine = engine;  
}  
}
```

Benefit:

Can use Petrol Engine, Diesel Engine, or Electric Engine without changing Car class. ---

What is a Dependency?

Anything a class needs to perform its work.

Examples:

- Repository
- DbContext
- Logger
- Email Service
- SMS Service
- Payment Service
- AI Service

ASP.NET Core Example

Interface

```
public interface IBookRepository
```

```
{  
    List<Book> GetAll();  
}
```

Implementation

```
public class BookRepository : IBookRepository
```

```
{  
    public List<Book> GetAll()  
    {  
        return new List<Book>();  
    }  
}
```

Registration

```
builder.Services.AddScoped<IBookRepository, BookRepository>();
```

Injection

```
public class BooksController : Controller
{
    private readonly IBookRepository _repo;
    public BooksController(IBookRepository repo)
    {
        _repo = repo;
    }
}
```

Common Places Where DI is Used

1. Repository Classes

- BookRepository
- EmployeeRepository

2. DbContext

- ApplicationDbContext

3. Logging

- ILogger<T>

4. Email Service

- IEmailService

5. SMS Service

- ISmsService

6. Payment Gateway

- RazorPay
- Stripe
- PayPal

7. Authentication Service

- IAuthService

8. AI Service

- IAIService

9. File Upload Service

- IFileService

10. Notification Service

- INotificationService

Types of Dependency Injection

1. Constructor Injection (Most Recommended)

```
public BooksController(IBookRepository repo)
{
}
```

2. Property Injection

```
public IBookRepository Repo { get; set; }
```

3. Method Injection

```
public void Save(IBookRepository repo)
{
}
```

Most Common:

Constructor Injection

DI Lifetimes

1. AddTransient

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

- New object every time

2. AddScoped

```
builder.Services.AddScoped<IBookRepository, BookRepository>(); -
```

One object per request

- Most commonly used

3. AddSingleton

```
builder.Services.AddSingleton<ICacheService, CacheService>();
```

- One object for entire application -----

Lifetime Comparison

Transient

- New object every time

- Lightweight services

Scoped

- One object per request
- Repositories, DbContext
- Most commonly used

Singleton

- One object for entire application
- Caching, Configuration

Interview Questions

Q. What is Dependency Injection?

DI is a design pattern where dependencies are provided from outside instead of creating them inside a class.

Q. Why do we need DI?

To achieve loose coupling, easy testing, easy maintenance, and scalability.

Q. What is a dependency?

Any object required by a class to perform its work.

Q. Which injection type is most commonly used?

Constructor Injection.

Q. Which lifetime is most commonly used in ASP.NET Core?

AddScoped().

Q. Difference between Transient, Scoped, and Singleton?

Transient:

New object every time.

Scoped:

One object per request.

Singleton:

One object for entire application.

One-Line Interview Answer

Dependency Injection is a design pattern that provides required objects to a class from outside instead of allowing the class to create them itself, making applications loosely coupled, maintainable, testable, and scalable.

DEPENDENCY INJECTION (DI) – SHORT NOTES

What is Dependency Injection?

Dependency Injection (DI) is a design pattern where required objects (dependencies) are provided to a class from outside instead of creating them inside the class.

Simple Definition:

"Don't create objects yourself, let ASP.NET Core provide them."

Why Do We Need DI?

Without DI:

- Tight Coupling
- Difficult Testing
- Difficult Maintenance
- Difficult to Replace Implementations

With DI:

- Loose Coupling
 - Easy Unit Testing
 - Easy Maintenance
 - Better Scalability
 - Easy to Replace Services
-

Real Life Example

Without DI

```
public class Car
{
    private Engine _engine = new Engine();
}
```

Problem:

Car is tightly coupled with Engine.

With DI

```
public class Car
{
    private readonly Engine _engine;
    public Car(Engine engine)
```

```
{  
    _engine = engine;  
}  
}
```

Benefit:

Can use Petrol Engine, Diesel Engine, or Electric Engine without changing Car class. ---

What is a Dependency?

Anything a class needs to perform its work.

Examples:

- Repository
- DbContext
- Logger
- Email Service
- SMS Service
- Payment Service
- AI Service

ASP.NET Core Example

Interface

```
public interface IBookRepository
```

```
{  
    List<Book> GetAll();  
}
```

Implementation

```
public class BookRepository : IBookRepository
```

```
{  
    public List<Book> GetAll()  
    {  
        return new List<Book>();  
    }  
}
```

Registration

```
builder.Services.AddScoped<IBookRepository, BookRepository>();
```

Injection

```
public class BooksController : Controller
{
    private readonly IBookRepository _repo;
    public BooksController(IBookRepository repo)
    {
        _repo = repo;
    }
}
```

Common Places Where DI is Used

1. Repository Classes

- BookRepository
- EmployeeRepository

2. DbContext

- ApplicationDbContext

3. Logging

- ILogger<T>

4. Email Service

- IEmailService

5. SMS Service

- ISmsService

6. Payment Gateway

- RazorPay
- Stripe
- PayPal

7. Authentication Service

- IAuthService

8. AI Service

- IAIService

9. File Upload Service

- IFileService

10. Notification Service

- INotificationService

Types of Dependency Injection

1. Constructor Injection (Most Recommended)

```
public BooksController(IBookRepository repo)
{
}
```

2. Property Injection

```
public IBookRepository Repo { get; set; }
```

3. Method Injection

```
public void Save(IBookRepository repo)
{
}
```

Most Common:

Constructor Injection

DI Lifetimes

1. AddTransient

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

- New object every time

2. AddScoped

```
builder.Services.AddScoped<IBookRepository, BookRepository>(); -
```

One object per request

- Most commonly used

3. AddSingleton

```
builder.Services.AddSingleton<ICacheService, CacheService>();
```

- One object for entire application -----

Lifetime Comparison

Transient

- New object every time

- Lightweight services

Scoped

- One object per request
- Repositories, DbContext
- Most commonly used

Singleton

- One object for entire application
- Caching, Configuration

Interview Questions

Q. What is Dependency Injection?

DI is a design pattern where dependencies are provided from outside instead of creating them inside a class.

Q. Why do we need DI?

To achieve loose coupling, easy testing, easy maintenance, and scalability.

Q. What is a dependency?

Any object required by a class to perform its work.

Q. Which injection type is most commonly used?

Constructor Injection.

Q. Which lifetime is most commonly used in ASP.NET Core?

AddScoped().

Q. Difference between Transient, Scoped, and Singleton?

Transient:

New object every time.

Scoped:

One object per request.

Singleton:

One object for entire application.

One-Line Interview Answer

Dependency Injection is a design pattern that provides required objects to a class from outside instead of allowing the class to create them itself, making applications loosely coupled, maintainable, testable, and scalable.

REPOSITORY PATTERN – SHORT NOTES

What is Repository Pattern?

Repository Pattern is a design pattern that acts as a middle layer between the application and the data source (Database, API, File, etc.).

Simple Definition:

"Repository is a data access layer that hides database code from the rest of the application."

Real Life Example

Imagine a restaurant.

Customer

|

V

Waiter

|

V

Kitchen

Customer does not go directly to the kitchen.

The waiter acts as a middleman.

MVC Example:

Controller = Customer

Repository = Waiter

Database = Kitchen

Without Repository Pattern

Controller directly talks to database.

```
public class BooksController
```

```
{
```

```
public IActionResult Index()
```

```
{
```

```
var books =
```

```
_dbContext.Books.ToList();
```

```
return View(books);
```

```
}
```

```
}
```

Problems:

- Database code inside Controller

- Difficult to maintain

- Difficult to test

- Tight coupling -----

----- With Repository

Pattern

Controller

|

V

Repository

|

V

Database

Controller only talks to Repository.

Repository talks to Database. -----

Repository Pattern Structure

Controller

|

V

IBookRepository

|

V

BookRepository

|

V

Database -----

----- Step 1: Create Model

```
public class Book
```

```
{
```

```
public int Id { get; set; }  
public string Title { get; set; }  
}
```

Step 2: Create Interface

```
public interface IBookRepository  
{  
    List<Book> GetAll();  
    Book GetById(int id);  
    void Add(Book book);  
    void Update(Book book);  
    void Delete(int id);  
}
```

Step 3: Create Repository

```
public class BookRepository  
: IBookRepository  
{  
    public List<Book> GetAll()  
    {  
        return _dbContext.Books.ToList();  
    }  
    public Book GetById(int id)  
    {  
        return _dbContext.Books  
            .FirstOrDefault(x => x.Id == id);  
    }  
    public void Add(Book book)  
    {  
        _dbContext.Books.Add(book);  
        _dbContext.SaveChanges();  
    }  
}
```

```

public void Update(Book book)
{
    _dbContext.Books.Update(book);
    _dbContext.SaveChanges();
}

public void Delete(int id)
{
    var book = GetById(id);
    _dbContext.Books.Remove(book);
    _dbContext.SaveChanges();
}
}

```

Step 4: Register Repository

```

builder.Services.AddScoped<
IBookRepository,
BookRepository>(); -----

```

----- Step 5: Use Repository in Controller

```

public class BooksController
{
    private readonly IBookRepository _repo;
    public BooksController(
        IBookRepository repo)
    {
        _repo = repo;
    }

    public IActionResult Index()
    {
        var books =
            _repo.GetAll();
        return View(books);
    }
}

```

}

Why Do We Need Repository Pattern?

Without Repository

Controller

|

Database

Every Controller writes SQL or EF code.

Messy.

With Repository

Controller

|

Repository

|

Database

Cleaner architecture.

Benefits of Repository Pattern

1. Separation of Concerns

Controller handles UI.

Repository handles data.

2. Loose Coupling

Controller depends on Interface.

Not Database.

3. Easy Testing

Can replace repository with fake data.

Example:

```
public class FakeBookRepository
```

```
: IBookRepository
```

```
{  
}
```

4. Reusability

Same repository can be used by:

- MVC
- Web API
- Background Jobs -----

5. Easier Maintenance

Database changes only affect Repository.

Controllers remain unchanged. -----

----- Common

Repository Methods

GetAll()

GetById()

Add()

Update()

Delete()

Exists()

Search() -----

----- BookStore Example

Get All Books

_repo.GetAll(); -----

Get Book By Id

_repo.GetById(5); -----

Add Book

_repo.Add(book); -----

----- Update Book

```
_repo.Update(book);
```

Delete Book

```
_repo.Delete(5);
```

Repository Pattern + Dependency Injection

Repository Pattern and DI usually work together.

Registration

```
builder.Services.AddScoped<
```

```
IBookRepository,
```

```
BookRepository>());
```

Injection

```
public BooksController(
```

```
IBookRepository repo)
```

```
{
```

```
_repo = repo;
```

```
}
```

Real World Examples

1. Book Repository

```
IBookRepository
```

2. Employee Repository

```
IEmployeeRepository
```

3. Customer Repository

```
ICustomerRepository
```

4. Product Repository

```
IProductRepository
```

5. Order Repository

IOrderRepository

Repository vs DbContext

DbContext

- Talks directly to database
- EF Core class

Repository

- Wrapper around DbContext
- Provides business-friendly methods

Example:

Instead of:

```
_dbContext.Books.ToList();
```

Use:

```
_repo.GetAll();
```

Cleaner code.

When Should We Use Repository Pattern?

- ✓ Medium Projects
- ✓ Large Projects
- ✓ Enterprise Applications
- ✓ Applications with Multiple Developers
- ✓ Applications requiring Unit Testing

When NOT To Use Repository Pattern?

Small Demo Projects

Small Proof of Concept Projects

Simple CRUD Applications

Sometimes EF Core DbContext is enough.

Industry Notes

Repository Pattern is commonly used in:

- ✓ Banking Applications
- ✓ ERP Systems

✓ Insurance Systems

✓ Healthcare Systems

✓ Enterprise Software

Interview Questions

Q. What is Repository Pattern?

Repository Pattern is a design pattern that creates a layer between the application and the database to manage data access operations.

Q. Why do we use Repository Pattern?

- Separation of Concerns
- Loose Coupling
- Easier Testing
- Better Maintainability

Q. What is the role of Repository?

Perform CRUD operations and hide database implementation details.

Q. Repository Pattern and Dependency Injection relationship?

Repository is usually registered in DI Container and injected into Controllers or Services.

Q. What methods typically exist in a Repository?

GetAll()

GetById()

Add()

Update()

Delete()

Q. Difference Between Repository and DbContext?

DbContext:

Direct database access.

Repository:

Abstraction layer over DbContext.

One-Line Interview Answer

Repository Pattern is a design pattern that creates a dedicated data access layer between the application and the database, improving maintainability, testability, and separation of concerns.

SINGLETON – SHORT NOTES

What is Singleton?

Singleton means only ONE object (instance) of a class is created and shared throughout the entire application.

Simple Definition:

"Create one object once and reuse it everywhere."

Real Life Example

Think of a company CEO.

There is only one CEO for the entire company.

All departments use the same CEO.

Not:

CEO for HR

CEO for Finance

CEO for Sales

Only one CEO.

Singleton works the same way.

Without Singleton

Request 1

Creates Object A

Request 2

Creates Object B

Request 3

Creates Object C

Memory Usage:

High

With Singleton

Application Starts

|

V

Create Object A

|

V

Reuse Same Object Everywhere

Request 1 -> Object A

Request 2 -> Object A

Request 3 -> Object A

Only one object exists. -----

----- ASP.NET Core Example

Registration

builder.Services.AddSingleton<

ICacheService,

CacheService>();

Now ASP.NET Core creates only one object. -

How Singleton Works

Application Starts

|

V

Create Object

|

V

Store In Memory

|

V

Reuse Same Object

For Entire Application -----

----- Example

```
public interface IMessageService
{
    void Send();
}

public class MessageService
: IMessageService
{
    public void Send()
    {
        Console.WriteLine("Message Sent");
    }
}
```

Registration

```
builder.Services.AddSingleton<
IMessageService,
MessageService>();
```

Now the same MessageService object is shared everywhere.

When Should We Use Singleton?

Use Singleton when:

- ✓ Data rarely changes
- ✓ Object creation is expensive
- ✓ Shared application-wide service
- ✓ Configuration data
- ✓ Caching
- ✓ Logging
- ✓ Application settings

Common Real-World Examples

1. Configuration Service

AppSettings

Connection Strings

API Keys

2. Cache Service

Product Cache

Country List Cache

State List Cache

3. Logging Service

ILogger

Serilog

NLog

4. AI Model Service

Loading AI model is expensive.

Load once.

Reuse everywhere.

5. Application Settings Service

Theme Settings

System Settings

Global Configuration

Why Singleton Improves Performance

Without Singleton

100 Requests

100 Objects Created

More Memory

More CPU

With Singleton

100 Requests

1 Object Created

Less Memory

Better Performance

When NOT To Use Singleton?

Do NOT use Singleton when data is user-specific.

Bad Examples:

- Shopping Cart
- Logged In User Data
- User Session Data
- Order Processing State

Reason:

All users would share the same object.

This can cause data leakage.

Problem Example

```
public class ShoppingCart
{
    public List<Product> Items
    {
        get;
        set;
    }
}
```

If Singleton:

User A adds Laptop

User B sees Laptop

Wrong behavior.

Singleton vs Scoped

Singleton

One Object

For Entire Application

Application Start

|

Same Object

|

Application Stop

Scoped

One Object

Per Request

Request 1 -> Object A

Request 2 -> Object B

Request 3 -> Object C

Most commonly used for:

DbContext

Repositories

Singleton vs Transient

Transient

New Object Every Time

Request:

Object A

Request:

Object B

Request:

Object C

Singleton

One Object Forever

Request:

Object A

Request:

Object A

Request:

Object A

Lifetime Comparison

Transient

- New object every time
- Lightweight services

Scoped

- One object per request
- DbContext, Repository
- Most commonly used

Singleton

- One object for entire application
 - Cache, Configuration, Logging
-

Interview Questions

Q. What is Singleton?

Singleton is a design pattern where only one instance of a class is created and shared throughout the entire application.

Q. When should Singleton be used?

When the same object can be safely shared across all users and requests.

Examples:

- Logging
 - Caching
 - Configuration
-

Q. When should Singleton NOT be used?

For user-specific data.

Examples:

- Shopping Cart
 - Session Data
 - User Information
-

Q. Difference between Singleton and Scoped?

Singleton:

One object for entire application.

Scoped:

One object per request.

Q. Difference between Singleton and Transient?

Singleton:

One object forever.

Transient:

New object every time.

One-Line Interview Answer

Singleton is a lifetime where only one object is created for the entire application and shared by all users and requests, making it useful for caching, logging, and configuration services.

ARCHITECTURE OF AN ASP.NET CORE MVC APPLICATION

ASP.NET Core MVC follows the MVC Architecture Pattern.

MVC = Model + View + Controller

MVC Architecture

Browser

|

V

Controller

|

V

Model / Repository / Database

|

V

Controller

|

V

View

|

V

Browser

Model

Represents application data.

Example:

Book

Employee

Customer

Order

Responsibilities:

- Store Data
 - Validation Rules
 - Business Data Representation
-

View

Responsible for UI.

Examples:

Index.cshtml

Create.cshtml

Edit.cshtml

Responsibilities:

- Display Data
 - Accept User Input
 - Show Validation Messages
-

Controller

Acts as mediator between Model and View.

Responsibilities:

- Receive Requests
- Call Business Logic
- Return View

Example:

BooksController

HomeController

FOLDER STRUCTURE

Project

|

+-- Controllers

|

+-- Models

|

+-- Views

|

+-- wwwroot

|

+-- Properties

|

+-- appsettings.json

|

+-- Program.cs

Controllers Folder

Contains all controllers.

Examples:

BooksController.cs

HomeController.cs

Purpose:

Handle user requests.

Models Folder

Contains model classes.

Examples:

Book.cs

Employee.cs

Purpose:

Represent application data.

Views Folder

Contains UI pages.

Example:

Views

|

+-- Books

| |

| +-- Index.cshtml

| +-- Create.cshtml

|

+-- Shared

|

+-- _Layout.cshtml

Purpose:

Display information to users.

Views/Shared Folder

Contains common views.

Examples:

_Layout.cshtml

_Error.cshtml

Purpose:

Reusable UI components.

wwwroot Folder

Contains static files.

Examples:

CSS

JavaScript

Images

Bootstrap

Purpose:

Public web content.

Properties Folder

Contains launch settings.

Example:

launchSettings.json

Purpose:

Application startup configuration.

CONFIGURATION FILES

1. Program.cs

Entry point of application.

Responsibilities:

- Configure Services
- Configure Middleware
- Configure Routing

Example:

```
builder.Services.AddControllersWithViews();
```

2. appsettings.json

Stores application configuration.

Examples:

Connection Strings

API Keys

Application Settings

Example:

```
{  
  "ConnectionStrings":  
  {
```

"DefaultConnection":

"Server=..."

}

}

3. launchSettings.json

Defines launch profiles.

Examples:

Application URL

Environment Variables -----

----- REQUEST FLOW IN MVC

User Request

|

V

Routing

|

V

Controller

|

V

Model/Repository

|

V

View

|

V

Browser Response -----

----- INTERVIEW QUESTIONS

Q. What are the three components of MVC?

Model, View, Controller

Q. What is the role of Controller?

Handle requests and return responses.

Q. What is the role of Model?

Represent application data.

Q. What is the role of View?

Display UI to users.

Q. What is Program.cs?

Entry point of ASP.NET Core application.

Q. What is appsettings.json?

Configuration file used to store application settings.

ONE-LINE INTERVIEW ANSWER

ASP.NET Core MVC architecture separates an application into Models, Views, and Controllers, providing a clean, maintainable, scalable, and testable structure for web applications. ACTION RESULT IN ASP.NET CORE MVC

What is Action Result?

ActionResult is a return type used by Controller Action Methods to send a response back to the browser.

Simple Definition:

"ActionResult tells MVC what response should be returned to the client."

Real Life Example

Customer asks waiter:

"I want coffee."

Waiter can respond with:

✓ Coffee

✓ Tea

✓ Sorry, Not Available

✓ Redirect to another counter

Similarly MVC Actions can return:

✓ View

✓JSON

✓Redirect

✓ File

✓ NotFound

✓ Content

All are Action Results. -----

----- Why Do We
Need ActionResult? Every request
expects a response. Example:

Browser Request

|

V

Controller Action

|

V

ActionResult

|

V

Browser Response -----

----- Basic Example

```
public IActionResult Index()
```

```
{
```

```
    return View();
```

```
}
```

Index() returns ViewResult. -----

----- Hierarchy

of ActionResult ActionResult

|

+-- ViewResult

+-- JsonResult

+-- RedirectResult

+-- RedirectToActionResult

+-- ContentResult
+-- FileResult
+-- NotFoundResult
+-- BadRequestResult
+-- OkResult

1. ViewResult

Most commonly used in MVC.

Returns Razor View.

Example:

```
public IActionResult Index()  
{  
    return View();  
}
```

MVC searches:

Views/Books/Index.cshtml

Passing Data To View

```
public IActionResult Index()  
{  
    var books =  
        _repo.GetAll();  
    return View(books);  
}
```

View receives:

@model List<Book>

When To Use ViewResult?

- ✓ Display Pages
- ✓ List Records
- ✓ Details Pages
- ✓ Forms

2. RedirectToActionResult

Redirects to another Action Method.

Example:

```
public IActionResult Create(Book book)
{
    _repo.Add(book);
    return RedirectToAction("Index");
}
```

Why Use RedirectToAction?

After successful save.

Pattern:

POST

|

Redirect

|

GET

Called PRG Pattern.

3. RedirectResult

Redirects to a URL.

Example:

```
return Redirect(
    "https://google.com");
```

Browser goes directly to URL.

Redirect vs RedirectToAction

Redirect()

Uses URL

RedirectToAction()

Uses Action Method

4. ContentResult

Returns plain text.

Example:

```
public IActionResult Hello()
{
    return Content("Welcome MVC");
}
```

Output:

Welcome MVC

When To Use ContentResult?

- ✓ Testing
 - ✓ Simple Messages
 - ✓ Debugging
-

5. JsonResult

Returns JSON Data.

Example:

```
public IActionResult GetBooks()
{
    return Json(
        _repo.GetAll());
}
```

Output:

```
[
  {
    "id":1,
    "title":"Clean Code"
  }
]
```

When To Use JsonResult?

- ✓ AJAX Calls
- ✓ APIs
- ✓ JavaScript Applications
- ✓ React/Angular Frontend

6. FileResult

Returns files.

Example:

```
public IActionResult Download()
{
    return File(
        "/Files/Book.pdf",
        "application/pdf");
}
```

Browser downloads file. -----

----- When To Use

FileResult?

✓ PDF Download

✓ Excel Export

✓ Images

✓ Documents -----

----- 7. NotFoundResult

Returns HTTP 404.

Example:

```
public IActionResult Details(int id)
{
    var book =
        _repo.GetByld(id);
    if(book == null)
        return NotFound();
    return View(book);
}
```

Output

404 Not Found

When To Use NotFound()?

✓ Record Not Found

✓ Invalid URL

✓ Missing Resource

8. BadRequestResult

Returns HTTP 400.

Example:

```
public IActionResult Details(int id)
{
    if(id <= 0)
        return BadRequest();
    return View();
}
```

Output

400 Bad Request

When To Use BadRequest()?

✓ Invalid Input

✓ Invalid Request

✓ Validation Failure

9. OkResult

Returns HTTP 200.

Example:

```
return Ok();
```

Output:

200 OK

Commonly used in Web APIs.

10. UnauthorizedResult

Returns HTTP 401.

Example:

```
return Unauthorized();
```

Meaning:

User not authenticated.

11. ForbidResult

Returns HTTP 403.

Example:

```
return Forbid();
```

Meaning:

User authenticated but not authorized.

ActionResult vs IActionResult

ActionResult

Old concrete class.

IActionResult

Interface introduced in ASP.NET Core.

Example:

```
public IActionResult Index()  
{  
    return View();  
}
```

Recommended.

Why Use IActionResult?

Because action can return multiple result types.

Example:

```

public IActionResult Details(int id)
{
    var book =
    _repo.GetByld(id);
    if(book == null)
    return NotFound();
    return View(book);
}

```

Possible Results:

ViewResult

NotFoundResult

Same method. -----

----- Real World BookStore

Example

List Books

```

public IActionResult Index()
{
    return View();
}

```

Returns:

ViewResult -----

----- Book Not Found

```

public IActionResult Details(int id)
{
    return NotFound();
}

```

Returns:

NotFoundResult -----

----- Save Book

```

public IActionResult Create(Book book)

```

```
{  
return RedirectToAction("Index");  
}
```

Returns:

RedirectToActionResult

API Response

```
public IActionResult GetBooks()  
{  
return Json(_repo.GetAll());  
}
```

Returns:

JsonResult

Most Commonly Used Action Results

1. View()

Display Page

2. RedirectToAction()

Redirect User

3. Json()

Return JSON

4. NotFound()

404 Error

5. BadRequest()

400 Error

6. File()

Download File

Interview Questions

Q. What is ActionResult?

ActionResult is a return type used by MVC Action Methods to return a response to the client.

Q. Why do we use IActionResult?

It allows an action method to return different types of responses.

Q. Difference between View() and RedirectToAction()?

View()

Returns a page.

RedirectToAction()

Redirects to another action.

Q. What does NotFound() return?

HTTP 404 Not Found.

Q. What does BadRequest() return?

HTTP 400 Bad Request.

Q. What is JsonResult?

Returns JSON data.

Q. What is FileResult?

Returns files to the browser.

One-Line Interview Answer

ActionResult (or IActionResult) is the return type of MVC Action Methods that determines what response—such as a View, JSON, Redirect, File, or HTTP Status Code—will be sent back to the client.

=====

MODELS & VIEWMODELS IN ASP.NET CORE MVC

What is a Model?

A Model represents application data.

It is used to store and transfer data within the application.

Simple Definition:

"Model represents the actual business data."

Real World Example

BookStore Application

Book Model

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
}
```

Book is a Model.

Responsibilities of Model

- ✓ Store Data
 - ✓ Represent Database Table
 - ✓ Validation Rules
 - ✓ Transfer Data
-

Examples of Models

Book

Employee

Customer

Order

Product

Student

Model and Database

Database Table

Books

Id

Title

Author

Price

Model

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
}
```

Usually one model maps to one database table.

What is a ViewModel?

A ViewModel is a class created specifically for a View.

Simple Definition:

"A ViewModel contains only the data required by the UI."

Why Do We Need ViewModel?

Suppose Book Model contains:

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
    public bool IsAdminOnly { get; set; }
    public DateTime CreatedDate { get; set; }
}
```

Do we want users to see all fields?

NO

We only want:

Title

Author

Price

So we create ViewModel.

Example ViewModel

```
public class BookViewModel
{
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
}
```

Contains only required UI data.

Model vs ViewModel

MODEL

Represents Business Data

Used by:

- Database
- Repository
- Services

Example:

Book

VIEWMODEL

Represents UI Data

Used by:

- View
- Forms
- Screens

Example:

BookViewModel

Real World Example

Registration Form

Database Table

User

Id

Name

Email

Password

Role

CreatedDate

Do we need all fields on screen?

NO

Create ViewModel

```
public class RegisterViewModel
{
    public string Name { get; set; }
    public string Email { get; set; }
    public string Password { get; set; }
    public string ConfirmPassword { get; set; }
}
```

Only UI fields.

Benefits of ViewModel

1. Security

Hide sensitive data.

Example:

Role

CreatedDate

Salary

PasswordHash

2. Better Validation

Different screens need different validations.

3. Better Performance

Only required data is sent.

4. Cleaner Views

UI receives exactly what it needs.

BookStore Example

Book Model

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
    public DateTime CreatedDate { get; set; }
}
```

Book Create ViewModel

```
public class BookCreateViewModel
{
    [Required]
    public string Title { get; set; }
    [Required]
    public string Author { get; set; }
    public decimal Price { get; set; }
}
```

Used only for Create Screen.

Book Edit ViewModel

```
public class BookEditViewModel
{
    public int Id { get; set; }
```

```
public string Title { get; set; }
public string Author { get; set; }
public decimal Price { get; set; }
}
```

Used only for Edit Screen.

Controller Example

```
public IActionResult Create()
{
    return View();
}

[HttpPost]
public IActionResult Create(
    BookCreateViewModel vm)
{
    if(ModelState.IsValid)
    {
        // Convert ViewModel to Model
        Book book = new Book
        {
            Title = vm.Title,
            Author = vm.Author,
            Price = vm.Price
        };
        _repo.Add(book);
        return RedirectToAction("Index");
    }
    return View(vm);
}
```

When To Use Model?

✓ Database Operations

- ✓ Entity Framework
 - ✓ Business Logic
 - ✓ Repository Layer
-

When To Use ViewModel?

- ✓ Create Form
 - ✓ Edit Form
 - ✓ Registration Screen
 - ✓ Login Screen
 - ✓ Dashboard Screen
 - ✓ Reports
-

Common Naming Convention

Book

BookViewModel

BookCreateViewModel

BookEditViewModel

BookDetailsViewModel

Interview Questions

Q. What is a Model?

A Model represents business data and usually maps to a database table.

Q. What is a ViewModel?

A ViewModel is a class designed specifically for a View and contains only the data required by the UI.

Q. Why do we need ViewModels?

- Security
 - Better Validation
 - Better Performance
 - Cleaner UI
-

Q. Difference between Model and ViewModel?

Model:

Represents actual business/database data.

ViewModel:

Represents screen/UI data.

Q. Can one Model have multiple ViewModels?

Yes.

Example:

Book Model

BookCreateViewModel

BookEditViewModel

BookDetailsViewModel

Q. Which is used with Entity Framework?

Model

Q. Which is used in Views?

ViewModel

One-Line Interview Answer

A Model represents actual business or database data, whereas a ViewModel is a UI-specific class that contains only the data required by a particular View, improving security, maintainability, and performance.

RAZOR VIEWS IN ASP.NET CORE MVC

What is Razor View?

Razor View is a UI page used in ASP.NET Core MVC to display data and generate HTML dynamically.

File Extension:

.cshtml

Simple Definition:

"Razor View is a combination of HTML and C# code."

Why Do We Need Razor Views?

Without Razor:

Only static HTML can be displayed.

With Razor:

- ✓ Display Dynamic Data
 - ✓ Loop Through Collections
 - ✓ Show Conditional Content
 - ✓ Create Forms
 - ✓ Display Model Data
-

What is Razor Syntax?

Razor Syntax allows us to write C# code inside HTML using @ symbol.

Example:

```
@DateTime.Now
```

Output:

Current Date and Time

Razor View Example

Index.cshtml

```
<h1>Welcome to BookStore</h1>
```

Output:

Welcome to BookStore

Displaying Dynamic Data

Controller

```
public IActionResult Index()
```

```
{
```

```
    ViewBag.Message =
```

```
    "Welcome MVC";
```

```
    return View();
```

```
}
```

View

```
<h2>@ViewBag.Message</h2>
```

Output:

Welcome MVC

Using Model in Razor View

Controller

```
public IActionResult Details()
```

```
{
```

```
    Book book = new Book
```

```
    {
```

```
        Title = "Clean Code"
```

```
    };
```

```
    return View(book);
```

```
}
```

View

```
@model Book
```

```
<h2>@Model.Title</h2>
```

Output:

Clean Code

Strongly Typed View

A View that receives a specific Model type.

Example:

```
@model Book
```

Benefits:

- ✓ IntelliSense

- ✓ Compile-Time Checking

- ✓ Type Safety

Displaying Multiple Records

Controller

```
return View(books);
```

View

```
@model List<Book>
@foreach(var book in Model)
{
<p>@book.Title</p>
}
```

Output:

Clean Code

Design Patterns

C# in Depth

Razor Expressions

Used to display values.

Syntax:

@expression

Example:

```
<p>@Model.Title</p>
```

Razor Code Block

Used for multiple lines of C# code.

Example:

```
@{
var message =
"Hello MVC";
}
<p>@message</p>
```

If Condition in Razor

```
@if(Model.Price > 1000)
{
<p>Premium Book</p>
}
else
{
```

<p>Standard Book</p>

} -----

- Loop in Razor

```
@for(int i = 1; i <= 5; i++)
```

```
{
```

```
<p>@i</p>
```

```
}
```

Output:

1

2

3

4

5 -----

-- Foreach Loop

```
@foreach(var book in Model)
```

```
{
```

```
<p>@book.Title</p>
```

```
}
```

Most commonly used. -----

----- Comments in

Razor

HTML Comment

```
<!-- HTML Comment -->
```

Visible in browser source. -----

----- Razor

Comment

```
@* Razor Comment *@
```

Not visible in browser source. -----

----- Layout

View

Provides common UI structure.

Example:

_Layout.cshtml

Contains:

✓ Header

✓ Navigation

✓ Footer

RenderBody()

Used inside Layout.

@RenderBody()

Placeholder where page content appears.

Partial View

Reusable UI component.

Example:

_SearchBar.cshtml

Use:

<partial name="_SearchBar" />

ViewData in Razor

Controller

ViewData["Title"] =

"Book Store";

View

<h2>@ViewData["Title"]</h2>

ViewBag in Razor

Controller

ViewBag.Title =

"Book Store";

View

<h2>@ViewBag.Title</h2>

Model vs ViewBag

Model

✓ Strongly Typed

✓ IntelliSense

✓ Compile-Time Checking

Recommended -----

----- ViewBag

✓ Dynamic

✗ No Type Safety

Use for small data only -----

----- Common

Razor Files

Index.cshtml

Details.cshtml

Create.cshtml

Edit.cshtml

Delete.cshtml

_Layout.cshtml

_ViewImports.cshtml

_ViewStart.cshtml -----

_ViewImports.cshtml

Used to import namespaces globally.

Example:

@using BookStore.Models -----

_ViewStart.cshtml

Used to define Layout.

Example:

@{

```
Layout = "_Layout";
```

```
}
```

Automatically applied to views.

Real World Example

Book List Page

```
@model List<Book>
```

```
<table>
```

```
@foreach(var book in Model)
```

```
{
```

```
<tr>
```

```
<td>@book.Title</td>
```

```
<td>@book.Author</td>
```

```
</tr>
```

```
}
```

```
</table>
```

Advantages of Razor Views

- ✓ Easy to Learn

- ✓ Clean Syntax

- ✓ Dynamic HTML

- ✓ Strongly Typed Views

- ✓ Built-in Security

- ✓ Tight MVC Integration

Interview Questions

Q. What is Razor View?

A Razor View is a .cshtml page that combines HTML and C# code to generate dynamic web pages.

Q. What is Razor Syntax?

Razor Syntax uses @ symbol to embed C# code inside HTML.

Q. What is a Strongly Typed View?

A View that receives a specific Model type using @model.

Q. What is the file extension of Razor Views?

.cshtml

Q. What is RenderBody()?

A placeholder inside Layout where page content is rendered.

Q. Difference between ViewBag and Model?

ViewBag:

Dynamic object.

Model:

Strongly typed object.

Q. What is _ViewStart.cshtml?

Used to define Layout for all views.

Q. What is _ViewImports.cshtml?

Used to import namespaces and tag helpers globally.

One-Line Interview Answer

Razor View is a .cshtml page in ASP.NET Core MVC that combines HTML and C# using Razor syntax (@) to generate dynamic web pages and display data from Controllers.

=====

HTML HELPER FUNCTIONS IN ASP.NET CORE MVC

What are HTML Helpers?

HTML Helpers are methods used inside Razor Views to generate HTML elements dynamically.

Simple Definition:

"HTML Helpers help us generate HTML controls using C# code."

Why Do We Need HTML Helpers?

Without HTML Helpers

```
<input type="text" name="Title" />
```

We manually write HTML.

With HTML Helpers

```
@Html.TextBox("Title")
```

MVC generates HTML automatically.

Benefits:

- ✓ Less Code
 - ✓ Strongly Typed
 - ✓ Easy Maintenance
 - ✓ Better Integration with Model
-

Types of HTML Helpers

1. Standard HTML Helpers
 2. Strongly Typed HTML Helpers
 3. Display Helpers
 4. Editor Helpers
-

1. TextBox Helper

Creates TextBox.

```
@Html.TextBox("Title")
```

Generated HTML

```
<input type="text" name="Title" />
```

TextBox with Default Value

```
@Html.TextBox(
```

```
"Title",
```

```
"Clean Code"
```

```
)
```

Output

```
<input type="text"
```

```
value="Clean Code" />
```

2. TextArea Helper

Creates multiline textbox.

```
@Html.TextArea("Description")
```

Output

```
<textarea></textarea>
```

3. Password Helper

Creates password field.

```
@Html.Password("Password")
```

Output

```
<input type="password" />
```

4. Hidden Helper

Creates hidden field.

```
@Html.Hidden("Id")
```

Output

```
<input type="hidden" />
```

Why Use Hidden Field?

Store values not visible to user.

Examples:

Id

Token

UserId

5. CheckBox Helper

```
@Html.CheckBox("IsActive")
```

Output

```
<input type="checkbox" />
```

6. RadioButton Helper

```
@Html.RadioButton(
```

```
"Gender",
```

```
"Male"
```

```
)
```

Output

```
<input type="radio" /> -----
```

```
----- 7.
```

DropDownList Helper

```
@Html.DropDownList("Category")
```

Output

```
<select>
```

```
</select> -----
```

----- Display Helpers

Used to display data. -----

```
----- 8. DisplayFor
```

```
@Html.DisplayFor(
```

```
model => model.Title
```

```
)
```

Displays value.

Output

Clean Code -----

```
----- 9. DisplayNameFor
```

```
@Html.DisplayNameFor(
```

```
model => model.Title
```

```
)
```

Output

Title -----

----- Editor Helpers

Automatically generate input controls.

```
-----
```

10. EditorFor

```
@Html.EditorFor(  
model => model.Title  
)
```

Output

```
<input type="text" />
```

11. LabelFor

Creates label.

```
@Html.LabelFor(  
model => model.Title  
)
```

Output

```
<label>Title</label>
```

12. TextBoxFor

Most Common Helper

```
@Html.TextBoxFor(  
model => model.Title  
)
```

Output

```
<input type="text"  
value="Clean Code" />
```

Why TextBoxFor is Better?

- ✓ Strongly Typed
 - ✓ IntelliSense
 - ✓ Compile-Time Checking
 - ✓ Model Binding Support
-

Validation Helpers

13. ValidationMessageFor

Displays validation error.

```
@Html.ValidationMessageFor(  
model => model.Title  
)
```

Output

Title is required.

14. ValidationSummary

Displays all validation errors.

```
@Html.ValidationSummary()
```

Book Create Example

Model

```
public class Book  
{  
    [Required]  
    public string Title  
{  
        get;  
        set;  
    }  
}
```

View

```
@model Book  
@using(Html.BeginForm())  
{  
    @Html.LabelFor(  
m => m.Title)  
    @Html.TextBoxFor(  
m => m.Title)  
    @Html.ValidationMessageFor(  

```

m => m.Title)

<button type="submit">

Save

</button>

} -----

- BeginForm()

Creates HTML Form.

@using(Html.BeginForm())

{

}

Generated HTML

<form method="post">

</form> -----

----- Commonly Used HTML

Helpers TextBoxFor()

LabelFor()

EditorFor()

DisplayFor()

ValidationMessageFor()

ValidationSummary()

DropDownListFor()

HiddenFor()

CheckBoxFor() -----

----- HTML Helpers vs Tag

Helpers

Old MVC

@Html.TextBoxFor(

m => m.Title) -----

----- ASP.NET Core

Preferred

<input asp-for="Title" />

Tag Helpers are now preferred.

Less code and easier to read.

Interview Questions

Q. What are HTML Helpers?

Methods used to generate HTML controls dynamically inside Razor Views.

Q. What is TextBoxFor?

Strongly typed helper used to create textbox controls.

Q. What is LabelFor?

Creates label for a model property.

Q. What is ValidationMessageFor?

Displays validation errors for a property.

Q. What is BeginForm()?

Creates an HTML form.

Q. Difference between HTML Helpers and Tag Helpers?

HTML Helpers:

@Html.TextBoxFor()

Tag Helpers:

<input asp-for="Title" />

ASP.NET Core prefers Tag Helpers.

One-Line Interview Answer

HTML Helpers are Razor methods that generate HTML elements such as textboxes, labels, dropdowns, and validation messages, making forms easier to create and strongly typed in ASP.NET Core MVC. =====

===== VIEWBAG, VIEWDATA AND TEMPDATA IN ASP.NET CORE MVC

Why Do We Need Them?

Sometimes we need to pass small amounts of data from Controller to View.

Examples:

✓ Success Messages

✓ Page Titles

✓ User Names

✓ Status Messages

ASP.NET Core provides:

1. ViewBag

2. ViewData

3. TempData

VIEWDATA

What is ViewData?

ViewData is a Dictionary object used to pass data from Controller to View.

Simple Definition:

"ViewData passes data from Controller to View using key-value pairs."

Controller

```
public IActionResult Index()
{
    ViewData["Message"] =
    "Welcome to MVC";
    return View();
}
```

View

```
<h2>@ViewData["Message"]</h2>
```

Output

Welcome to MVC

Characteristics of ViewData

✓ Dictionary Based

✓ Key-Value Pair

✓ Controller → View

✓ Available only for current request

✗ Requires Type Casting

Example

Controller

```
ViewData["Age"] = 25;
```

View

```
@ViewData["Age"]
```

Output

25

VIEWBAG

What is ViewBag?

ViewBag is a dynamic object used to pass data from Controller to View.

Simple Definition:

"ViewBag is a wrapper around ViewData."

Controller

```
public IActionResult Index()
```

```
{
```

```
    ViewBag.Message =
```

```
    "Welcome MVC";
```

```
    return View();
```

```
}
```

View

```
<h2>@ViewBag.Message</h2>
```

Output

Welcome MVC

Characteristics of ViewBag

- ✓ Dynamic Object
- ✓ No Type Casting Required
- ✓ Easy Syntax
- ✓ Controller → View
- ✓ Current Request Only

Example

Controller

ViewBag.Name = "Sumit";

View

@ViewBag.Name

Output

Sumit

VIEWDATA vs VIEWBAG

ViewData

ViewData["Name"]

Dictionary Based

Requires Key

ViewBag

ViewBag.Name

Dynamic Object

Easy Syntax

Internally

ViewBag uses ViewData.

These two are almost the same.

TEMPDATA

What is TempData?

TempData is used to pass data from one request to another.

Simple Definition:

"TempData survives Redirects."

Why Do We Need TempData?

ViewBag and ViewData work only for the current request.

TempData works across requests.

Example

Create Action

[HttpPost]

```
public IActionResult Create(Book book)
```

```
{
```

```
    TempData["Success"] =
```

```
    "Book Created Successfully";
```

```
    return RedirectToAction("Index");
```

```
}
```

Index Action

```
public IActionResult Index()
```

```
{
```

```
    return View();
```

```
}
```

View

```
<h2>@TempData["Success"]</h2>
```

Output

Book Created Successfully

Most Common Use of TempData

✓ Success Message

✓ Error Message

✓ Warning Message

✓ Redirect Information

TempData Lifecycle

Request 1

TempData Set

|

Redirect

|

Request 2

|

TempData Available

After Reading

TempData Removed Automatically

Keep TempData

TempData.Keep();

Prevents deletion.

Peek TempData

TempData.Peek("Success");

Reads value without removing it.

Real World Example

Book Created

Redirect To Index

Show Message:

"Book Created Successfully"

Perfect use case for TempData.

Comparison

VIEWDATA

✓ Controller → View

✓ Current Request Only

✗ Lost After Redirect

VIEWBAG

✓ Controller → View

✓ Current Request Only

✗ Lost After Redirect

TEMPDATA

✓ Controller → View

✓ Survives Redirect

✓ Next Request Available

Memory Trick

ViewData

Current Request

Dictionary

ViewBag

Current Request

Dynamic

TempData

Next Request

Redirect Friendly

When To Use What?

Use ViewData

✓ Small Data

✓ Page Titles

✓ Labels

Use ViewBag

✓ Messages

✓ User Name

✓ Small Dynamic Data

Use TempData

✓ Success Messages

✓ Error Messages

✓ Redirect Scenarios

BookStore Example

ViewData

ViewData["Title"] =

"Book List";

ViewBag

ViewBag.UserName =

"Admin";

TempData

TempData["Success"] =

"Book Saved Successfully";

Interview Questions

Q. What is ViewData?

ViewData is a dictionary used to pass data from Controller to View.

Q. What is ViewBag?

ViewBag is a dynamic wrapper around ViewData.

Q. What is TempData?

TempData is used to pass data between requests and survives redirects.

Q. Difference between ViewBag and ViewData?

ViewData:

Dictionary based.

ViewBag:

Dynamic object.

Q. Difference between ViewBag and TempData?

ViewBag:

Current Request Only.

TempData:

Available after Redirect.

Q. Which one is used for Success Messages?

TempData

Q. Which one survives Redirect?

TempData

One-Line Interview Answer

ViewData and ViewBag are used to pass data from Controller to View for the current request, while TempData is used to pass data between requests and survives redirects.

===== VALIDATION USING
DATA ANNOTATIONS IN ASP.NET CORE MVC

What is Validation?

Validation is the process of checking whether user input is correct and acceptable before saving it.

Simple Definition:

"Validation ensures that only valid data enters the application."

Why Do We Need Validation?

Users can enter:

Empty Name

■ Negative Price

■ Invalid Email

■ Wrong Mobile Number

■

Validation prevents bad data.

Real World Example

BookStore Application

User enters:

Title = ""

Price = -500

Without Validation:

Invalid data gets saved.

With Validation:

User gets error message.

What are Data Annotations?

Data Annotations are attributes used to define validation rules on Model properties.

Example:

[Required]

```
public string Title { get; set; }
```

Benefits of Data Annotations

- ✓ Easy to Use
 - ✓ Built-in Validation
 - ✓ Less Code
 - ✓ Server-Side Validation
 - ✓ Client-Side Validation Support
-

Model Example

```
public class Book
```

```
{
```

```
public int Id { get; set; }
```

```
[Required]
```

```
public string Title { get; set; }
```

```
public decimal Price { get; set; }
```

```
}
```

1. Required Attribute

Makes field mandatory.

Example:

[Required]

```
public string Title { get; set; }
```

Error:

"The Title field is required." -----

2. StringLength Attribute

Controls minimum and maximum length.

Example:

[StringLength(50)]

```
public string Title { get; set; }
```

Maximum 50 characters allowed. -----

----- Minimum and

Maximum Length [StringLength(

50,

MinimumLength = 5)]

```
public string Title { get; set; }
```

Title must be between 5 and 50 characters. -

3. Range Attribute

Used for numeric validation.

Example:

[Range(1,10000)]

```
public decimal Price { get; set; }
```

Valid:

100

500

1000

Invalid:

0

-10

4. EmailAddress Attribute

Validates email format.

[EmailAddress]

```
public string Email { get; set; }
```

Valid:

abc@gmail.com

Invalid:

abc

5. Phone Attribute

Validates phone number.

[Phone]

```
public string Mobile { get; set; }
```

6. Url Attribute

Validates website URL.

[Url]

```
public string Website { get; set; }
```

Valid:

https://google.com

7. Compare Attribute

Compares two fields.

Example:

Password Confirmation

[Compare("Password")]

```
public string ConfirmPassword
```

```
{
```

```
get;
```

```
set;
```

```
}
```

Both values must match.

8. RegularExpression Attribute

Used for custom validation.

Example:

Only 10-digit mobile number.

```
[RegularExpression(
```

```
@^[0-9]{10}$")]
```

```
public string Mobile
```

```
{
```

```
get;
```

```
set;
```

```
}
```

9. Display Attribute

Changes display name.

```
[Display(Name="Book Title")]
```

```
public string Title
```

```
{
```

```
get;
```

```
set;
```

```
}
```

Error Message:

Book Title is required.

Custom Error Message

```
[Required(
```

```
ErrorMessage =
```

```
"Book Title is mandatory")]
```

```
public string Title
```

```
{  
get;  
set;  
}
```

Output:

Book Title is mandatory

Complete Example

```
public class Book  
{  
    public int Id { get; set; }  
    [Required(  
        ErrorMessage =  
        "Title is required")]  
    public string Title  
    {  
        get;  
        set;  
    }  
    [Required]  
    public string Author  
    {  
        get;  
        set;  
    }  
    [Range(1,10000)]  
    public decimal Price  
    {  
        get;  
        set;  
    }  
}
```

How MVC Performs Validation?

User Submits Form

I

V

Model Binding

I

V

Data Annotation Validation

I

V

ModelState Updated

I

V

Valid or Invalid

What is ModelState?

ModelState stores validation results.

Example:

```
if(ModelState.IsValid)
```

```
{
```

```
// Save Data
```

```
}
```

Controller Example

```
[HttpPost]
```

```
public IActionResult Create(Book book)
```

```
{
```

```
if(ModelState.IsValid)
```

```
{
```

```
_repo.Add(book);
```

```
return RedirectToAction(
```

```
"Index");
```

```
}  
return View(book);  
}
```

Validation Message in View

```
@Html.ValidationMessageFor(  
m => m.Title)
```

Displays:

Title is required. -----

----- Validation Summary

```
@Html.ValidationSummary()
```

Displays all validation errors. -----

----- Tag Helper

Validation

```
<input asp-for="Title" />
```

```
<span asp-validation-for="Title">
```

```
</span> -----
```

----- Client-Side Validation

Validation happens in browser.

Benefits:

✓ Faster

✓ Better User Experience -----

----- Server-Side

Validation

Validation happens on server.

Most Important.

Never trust client-side validation alone.

Best Practice

Always use:

✓ Client-Side Validation

AND

✓ Server-Side Validation

Common Data Annotation Attributes

[Required]

[StringLength]

[Range]

[EmailAddress]

[Phone]

[Url]

[Compare]

[RegularExpression]

[Display]

Real World Examples

Registration Form

Email Validation

Password Validation

Confirm Password

Employee Management

Name Required

Salary Range

Email Format

BookStore

Title Required

Author Required

Price Range

Interview Questions

Q. What is Validation?

Validation is the process of ensuring user input is correct before processing or saving. -----

Q. What are Data Annotations?

Attributes used to define validation rules on model properties.

Q. What is Required Attribute?

Makes a field mandatory.

Q. What is Range Attribute?

Validates numeric values within a specified range.

Q. What is Compare Attribute?

Compares two properties.

Mostly used for Confirm Password.

Q. What is ModelState?

ModelState stores validation results.

Q. What does ModelState.IsValid do?

Checks whether all validation rules passed successfully.

Q. Which validation is most important?

Server-Side Validation.

One-Line Interview Answer

Data Annotations are attributes applied to Model properties to perform validation such as Required, Range, Email, and StringLength, ensuring that only valid data is accepted by the application.

===== SELF
VALIDATED MODEL IN ASP.NET CORE MVC

What is a Self Validated Model?

A Self Validated Model is a model that contains its own custom validation logic instead of relying only on Data Annotations.

Simple Definition:

"A model validates itself by implementing custom validation rules." -----

Why Do We Need Self Validation?

Data Annotations work well for simple validation.

Examples:

[Required]

[Range]

[EmailAddress]

But some business rules are complex.

Example:

Book Price must be greater than Discount Price.

Start Date must be before End Date.

Age must be greater than 18 if Country = India.

These validations cannot be handled easily using standard Data Annotations. --

How Self Validation Works?

ASP.NET Core provides:

IDataValidatableObject

Interface

A model implements this interface and provides custom validation logic. -----

Syntax

public class Book

: IDataValidatableObject

{

}

Example

Book Price Validation

Business Rule:

Price must be greater than 0.

public class Book

```

: IValidatableObject
{
    public string Title
    {
        get;
        set;
    }
    public decimal Price
    {
        get;
        set;
    }
    public IEnumerable<ValidationResult>
    Validate(
        ValidationContext validationContext)
    {
        if(Price <= 0)
        {
            yield return new ValidationResult(
                "Price must be greater than zero",
                new[]
                {
                    nameof(Price)
                });
        }
    }
}

```

What is ValidationResult?

ValidationResult represents a validation error.

Example:

```

yield return new ValidationResult(

```

```
"Price must be greater than zero");
```

Multiple Validation Rules

```
public IEnumerable<ValidationResult>
```

```
Validate(
```

```
ValidationContext validationContext)
```

```
{
```

```
if(Price <= 0)
```

```
{
```

```
yield return new ValidationResult(
```

```
"Price must be greater than zero",
```

```
new[]
```

```
{
```

```
nameof(Price)
```

```
});
```

```
}
```

```
if(string.IsNullOrEmpty(Title))
```

```
{
```

```
yield return new ValidationResult(
```

```
"Title is required",
```

```
new[]
```

```
{
```

```
nameof(Title)
```

```
});
```

```
}
```

```
}
```

Real World Example

BookStore Application

Rule:

Discount Price must be less than Book Price.

```
public class Book
```

```

: IValidatableObject
{
    public decimal Price
    {
        get;
        set;
    }
    public decimal DiscountPrice
    {
        get;
        set;
    }
    public IEnumerable<ValidationResult>
    Validate(
        ValidationContext validationContext)
    {
        if(DiscountPrice >= Price)
        {
            yield return new ValidationResult(
                "Discount Price must be less than Price",
                new[]
                {
                    nameof(DiscountPrice)
                });
        }
    }
}

```

Employee Example

Business Rule:

Age must be at least 18.

```
public class Employee
```

```

: IValidatableObject
{
    public int Age
    {
        get;
        set;
    }
    public IEnumerable<ValidationResult>
    Validate(
        ValidationContext validationContext)
    {
        if(Age < 18)
        {
            yield return new ValidationResult(
                "Employee must be at least 18 years old",
                new[]
                {
                    nameof(Age)
                });
        }
    }
}

```

How MVC Executes Self Validation?

User Submits Form

|

V

Model Binding

|

V

Data Annotation Validation

|

V

IValidatableObject Validation

I

V

ModelState Updated

I

V

Controller Executes

Controller Example

[HttpPost]

public IActionResult Create(Book book)

{

if(ModelState.IsValid)

{

// Save Data

return RedirectToAction(

"Index");

}

return View(book);

}

Can We Use Data Annotations and Self Validation Together?

YES

Most common approach.

Example:

public class Book

: IValidatableObject

{

[Required]

public string Title

{

get;

```

set;
}
public decimal Price
{
get;
set;
}
public decimal DiscountPrice
{
get;
set;
}
public IEnumerable<ValidationResult>
Validate(
ValidationContext validationContext)
{
if(DiscountPrice >= Price)
{
yield return new ValidationResult(
"Discount Price must be less than Price");
}
}
}

```

When Should We Use Self Validation?

- ✓ Complex Business Rules
- ✓ Multiple Property Validation
- ✓ Cross Field Validation
- ✓ Custom Business Logic

Examples

Book Price vs Discount Price

Start Date vs End Date

Password vs Confirm Password

Minimum Age Requirement

Salary vs Bonus Validation

Advantages

✓ Powerful Validation

✓ Centralized Rules

✓ Easy Maintenance

✓ Business Rule Enforcement

Interview Questions

Q. What is a Self Validated Model?

A model that contains its own validation logic by implementing IValidatableObject.

Q. Which interface is used for Self Validation?

IValidatableObject

Q. Which method must be implemented?

Validate()

Q. When should Self Validation be used?

When validation depends on multiple properties or complex business rules.

Q. Can Data Annotations and Self Validation be used together?

Yes.

Data Annotations handle simple validation.

Self Validation handles complex business rules.

One-Line Interview Answer

A Self Validated Model is a model that implements IValidatableObject and contains its own custom validation logic, allowing complex business rules and cross-property validations that cannot be handled by standard Data Annotations alone.

===== STRONGLY TYPED
VIEWS IN ASP.NET CORE MVC

What is a Strongly Typed View?

A Strongly Typed View is a view that is bound to a specific Model type.

Simple Definition:

"A Strongly Typed View knows exactly what type of data it will receive."

Why Do We Need Strongly Typed Views?

Suppose Controller sends a Book object.

```
public IActionResult Details()  
{  
    Book book = new Book  
{  
        Title = "Clean Code",  
        Author = "Robert Martin"  
    };  
    return View(book);  
}
```

The View should know that it is receiving a Book object.

Strongly Typed Views provide this information.

How to Create a Strongly Typed View?

Use @model directive.

Example:

```
@model Book
```

This tells MVC:

"This View will receive a Book object."

Controller Example

```
public IActionResult Details()  
{  
    Book book = new Book  
{  
        Title = "Clean Code",
```

```
Author = "Robert Martin"
```

```
};
```

```
return View(book);
```

```
} -----
```

- View Example

```
@model Book
```

```
<h2>@Model.Title</h2>
```

```
<p>@Model.Author</p>
```

Output

Clean Code

```
Robert Martin -----
```

```
----- What is Model?
```

In a Strongly Typed View:

```
@model Book
```

MVC automatically creates:

Model

Object

Type:

```
Book -----
```

```
----- Accessing Properties
```

```
@model Book
```

```
<h2>@Model.Title</h2>
```

```
<p>@Model.Author</p>
```

```
<p>@Model.Price</p> -----
```

```
----- Strongly Typed
```

View with List Controller

```
public IActionResult Index()
```

```
{
```

```
List<Book> books =  
_repo.GetAll();  
return View(books);  
}
```

View

```
@model List<Book>  
@foreach(var book in Model)  
{  
<p>@book.Title</p>  
}
```

Output

Clean Code

Design Patterns

C# in Depth

Why Strongly Typed Views are Better?

1. IntelliSense Support

As soon as you type:

@Model.

Visual Studio shows:

Title

Author

Price

Id

2. Compile-Time Checking

Wrong Property

@Model.BookName

Compiler Error

Instead of Runtime Error.

3. Type Safety

MVC knows exactly what object is expected.

4. Easier Maintenance

Refactoring becomes easier. -----

----- Strongly Typed View

vs ViewBag

ViewBag Example

Controller

ViewBag.Title =

"Clean Code";

View

@ViewBag.Title -----

----- Problems with ViewBag

✗ No IntelliSense

✗ No Compile-Time Checking

✗ Runtime Errors Possible -----

----- Strongly Typed Example

Controller

return View(book);

View

@model Book

@Model.Title -----

----- Benefits

✓ IntelliSense

✓ Type Safety

✓ Compile-Time Validation -----

----- Strongly Typed View vs

ViewData

ViewData

ViewData["Title"]

Dictionary Based

Strongly Typed View

@Model.Title

Object Based

Recommended Approach

For Small Data

ViewBag

ViewData

For Business Data

Always Use:

Strongly Typed Views

BookStore Example

Book Model

```
public class Book
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Author { get; set; }
    public decimal Price { get; set; }
}
```

Controller

```
public IActionResult Details()
```

```
{
    Book book =
    new Book
    {
```

```

Id = 1,
Title = "Clean Code",
Author = "Robert Martin",
Price = 999
};
return View(book);
}

```

```

-----
View
@model Book
<h1>@Model.Title</h1>
<p>@Model.Author</p>
<p>@Model.Price</p> -----

```

----- Strongly Typed Create

```

View
@model Book
<form method="post">
<input asp-for="Title" />
<input asp-for="Author" />
<input asp-for="Price" />
<button type="submit">
Save
</button>
</form> -----

```

----- Benefits with Tag Helpers

asp-for works only with Model properties.

Provides:

- ✓ IntelliSense
 - ✓ Validation Integration
 - ✓ Strong Typing -----
-

Commonly Used Model Types

Single Object

@model Book

Collection

@model List<Book>

ViewModel

@model BookCreateViewModel

Dashboard Model

@model DashboardViewModel

Interview Questions

Q. What is a Strongly Typed View?

A View that is bound to a specific Model type using the @model directive. --

Q. How do you create a Strongly Typed View?

Using:

@model Book

Q. What are the advantages of Strongly Typed Views?

✓ IntelliSense

✓ Type Safety

✓ Compile-Time Checking

✓ Easier Maintenance

Q. Difference between Strongly Typed View and ViewBag?

Strongly Typed View:

Uses Model object.

ViewBag:

Uses dynamic properties.

Q. Which approach is recommended in MVC?

Strongly Typed Views.

Q. Can a Strongly Typed View receive a List?

Yes.

Example:

@model List<Book>

One-Line Interview Answer

A Strongly Typed View is a Razor View that is bound to a specific Model type using the @model directive, providing IntelliSense, type safety, and compile-time checking for better maintainability and development experience.

=====

PARTIAL VIEW IN ASP.NET CORE MVC

What is a Partial View?

A Partial View is a reusable Razor View that can be used inside multiple Views.

Simple Definition:

"A Partial View is a small reusable piece of UI."

Why Do We Need Partial Views?

Imagine a BookStore application.

Pages:

Home Page

Books Page

Authors Page

Categories Page

All pages contain:

✓ Search Box

✓ Navigation Menu

✓ Login Panel

✓ Footer

Without Partial View:

Copy-Paste Same HTML Everywhere

Problems:

- Duplicate Code
- Difficult Maintenance
- More Errors

With Partial View

Create Once

Reuse Everywhere

Benefits:

- ✓ Reusable UI
- ✓ Cleaner Code
- ✓ Easy Maintenance
- ✓ Better Organization

Real World Example

Amazon Website

Search Bar

Navigation Menu

Shopping Cart Icon

Login Panel

These are good candidates for Partial Views.

Partial View Architecture

Main View

|

+----- Partial View

|

+----- Partial View

|

+----- Partial View

Examples of Partial Views

_SearchBar.cshtml

_CategoryMenu.cshtml

_LoginPanel.cshtml

_BookCard.cshtml

Naming Convention

Partial Views usually start with underscore (_)

Examples:

_SearchBar.cshtml

_BookCard.cshtml

_Menu.cshtml

Where are Partial Views Stored?

Views

|

+-- Shared

|

+-- _SearchBar.cshtml

+-- _BookCard.cshtml

Most commonly:

Views/Shared

Creating Partial View

_SearchBar.cshtml

<form>

<input

placeholder="Search Books" />

<button>

Search

</button>

</form>

Using Partial View

Inside another View

```
<partial name="_SearchBar" /> -----
```

Output

Search Box appears on page. -----

----- BookStore Example

_BookCard.cshtml

@model Book

```
<div class="card">
```

```
<h3>@Model.Title</h3>
```

```
<p>@Model.Author</p>
```

```
<p>@Model.Price</p>
```

```
</div>
```

Using BookCard

```
<partial
```

```
name="_BookCard"
```

```
model="book" /> -----
```

MVC sends current book object to Partial View. -

Passing Data to Partial View

Main View

```
@foreach(var book in Model)
```

```
{
```

```
<partial
```

```
name="_BookCard"
```

```
model="book" />
```

```
}
```

Partial View Receives

@model Book

Now it can access:

@Model.Title

@Model.Author

@Model.Price

Partial View vs View

View

Complete Page

Examples:

Index.cshtml

Create.cshtml

Edit.cshtml

Partial View

Small Reusable UI Component

Examples:

_SearchBar.cshtml

_Menu.cshtml

_BookCard.cshtml

Partial View vs Layout

LAYOUT

Entire Page Structure

Header

Navigation

Footer

PARTIAL VIEW

Small UI Components

Search Bar

Login Panel

Menu

Example

Layout

Header

Navigation

@RenderBody()

Footer

Partial View

Search Bar

Category Menu

Book Card

Benefits of Partial Views

1. Reusability

Create once.

Use many times.

2. Easy Maintenance

Change one file.

All pages updated.

3. Less Code Duplication

Avoid copy-paste.

4. Better Readability

Views become cleaner.

Without Partial View

Index.cshtml

500 lines

Create.cshtml

500 lines

Edit.cshtml

500 lines

With Partial View

Index.cshtml

100 lines

Create.cshtml

100 lines

Edit.cshtml

100 lines

Reusable components separated.

Real World Usage

✓ Search Bar

✓ Navigation Menu

✓ Login Widget

✓ Shopping Cart

✓ Product Card

✓ Footer

✓ Header

✓ User Profile Panel

Interview Questions

Q. What is a Partial View?

A Partial View is a reusable Razor View used to render a portion of UI inside another View.

Q. Why do we use Partial Views?

To avoid duplicate UI code and improve maintainability.

Q. How do you create a Partial View?

Create a .cshtml file, usually starting with "_".

Example:

_SearchBar.cshtml

Q. How do you render a Partial View?

<partial name="_SearchBar" />

Q. Can Partial Views receive Model data?

Yes.

Example:

<partial
name="_BookCard"
model="book" />

Q. Difference between View and Partial View?

View:

Complete Page.

Partial View:

Reusable UI Component.

Q. Difference between Layout and Partial View?

Layout:

Entire Page Template.

Partial View:

Small Reusable Section of UI.

When Should We Use Partial Views?

✓ Repeated UI Components

✓ Search Bars

✓ Menus

✓ Product Cards

✓ Login Panels

✓ Reusable Sections

One-Line Interview Answer

A Partial View is a reusable Razor View that renders a portion of UI inside another View, helping reduce code duplication and improve maintainability in ASP.NET Core MVC applications.

===== ROUTING ENGINE &
ROUTING TABLE IN ASP.NET CORE MVC

What is Routing?

Routing is the process of mapping an incoming URL to a specific Controller and Action Method.

Simple Definition:

"Routing decides which Controller and Action should handle a user request."

Real World Example

Suppose user enters URL:

`https://localhost:5001/Books/Details/5`

Routing determines:

Controller = Books

Action = Details

Id = 5

And executes:

```
public IActionResult Details(int id)
{
}
```

Why Do We Need Routing?

Without Routing:

MVC would not know:

✓ Which Controller to Execute

✓ Which Action to Call

✓ Which Data to Pass

Routing acts like a traffic police.

What is Routing Engine?

Routing Engine is the component inside ASP.NET Core that analyzes the URL and finds the correct Controller and Action Method.

Simple Definition:

"Routing Engine matches incoming URLs with routes defined in the application."

Request Flow

Browser Request

|

V

Routing Engine

|

V

Routing Table

|

V

Controller

|

V

Action Method

|

V

Response

What is Routing Table?

Routing Table is a collection of route definitions maintained by ASP.NET Core.

Simple Definition:

"Routing Table contains all URL patterns used by the application."

Default Route

Program.cs

app.MapControllerRoute(

name: "default",

pattern:

```
"{controller=Home}/{action=Index}/{id?}";
```

This route is added to Routing Table.

Understanding Default Route

Pattern:

```
{controller=Home}/{action=Index}/{id?}
```

controller

Controller Name

Default:

Home

action

Action Method Name

Default:

Index

id?

Optional Parameter

? means optional

Examples

URL:

/

Controller = Home

Action = Index

URL:

/Books

Controller = Books

Action = Index

URL:

/Books/Details

Controller = Books

Action = Details

URL:

/Books/Details/5

Controller = Books

Action = Details

Id = 5

How Routing Engine Works

Step 1

User sends request:

/Books/Details/5

Step 2

Routing Engine reads URL.

Step 3

Checks Routing Table.

Step 4

Finds matching route.

Step 5

Extracts values:

Controller = Books

Action = Details

Id = 5

Step 6

Calls:

BooksController

Details(5)

Routing Table Example

Route 1

{controller=Home}/{action=Index}/{id?} -----

Route 2

Products/{id}

Route 3

Books/Details/{id} -----

These routes are stored internally in Routing Table. -

Conventional Routing

Most common routing type.

Program.cs

app.MapControllerRoute(
name: "default",
pattern:

"{controller=Home}/{action=Index}/{id?}");

URL structure follows convention. -----

Attribute Routing

Route defined directly on action.

[Route("books/all")]

public IActionResult Index()
{
}

URL:

/books/all

Route Parameters

Values passed through URL.

Example:

/Books/Details/10

10 is Route Parameter.

Action:

```
public IActionResult Details(int id)
{
}
```

MVC automatically assigns:

id = 10 -----

----- Route Values

URL:

/Books/Edit/7

Routing Engine Extracts:

Controller = Books

Action = Edit

Id = 7 -----

----- Optional Parameters

Pattern:

{id?}

? means parameter is optional.

Example:

/Books/Details

Works -----

----- /Books/Details/5

Also Works -----

Custom Route Example

```
app.MapControllerRoute(
```

```
name: "books",
```

```
pattern:
```

```
"BookInfo/{id}",
```

```
defaults:
```

```
new
```

```
{
```

```
controller = "Books",
```

```
action = "Details"
```

```
});
```

URL:

/BookInfo/5

Calls:

BooksController

Details(5)

Routing Terminology

URL

Incoming request path.

Route

Pattern used to match URL.

Route Parameter

Value inside URL.

Routing Table

Collection of routes.

Routing Engine

Component that matches URLs to routes.

Real World Example

Amazon Product Page

/product/123

Routing Engine Extracts:

Controller = Product

Action = Details

Id = 123

BookStore Example

/books/details/5

Routing Engine Extracts:

Controller = Books

Action = Details

Id = 5

Advantages of Routing

- ✓ Clean URLs
 - ✓ SEO Friendly
 - ✓ Easy Navigation
 - ✓ Better User Experience
 - ✓ Flexible URL Structure
-

Interview Questions

Q. What is Routing?

Routing is the process of mapping URLs to Controller Actions.

Q. What is Routing Engine?

Routing Engine is the ASP.NET Core component that matches incoming URLs with routes and executes the correct Controller Action.

Q. What is Routing Table?

Routing Table is a collection of route definitions maintained by ASP.NET Core.

Q. What is the default route in ASP.NET Core MVC?

`{controller=Home}/{action=Index}/{id?}`

Q. What does id? mean?

Optional route parameter.

Q. Difference between Conventional Routing and Attribute Routing?

Conventional Routing:

Defined in Program.cs

Attribute Routing:

Defined using [Route] attribute on Controller/Action.

Q. What is a Route Parameter?

A value passed through URL.

Example:

`/Books/Details/5`

5 is Route Parameter.

One-Line Interview Answer

The Routing Engine matches incoming URLs against the Routing Table and determines which Controller and Action Method should handle the request, enabling clean and flexible URL navigation in ASP.NET Core MVC.

===== 404 ERROR
(RESOURCE NOT FOUND) IN ASP.NET CORE MVC

What is 404 Error?

404 (Not Found) is an HTTP Status Code that indicates the requested resource could not be found on the server.

Simple Definition:

"The user requested something that does not exist."

HTTP Status Code

404 = Not Found

Meaning:

Server is running.

Request reached server.

But requested resource was not found.

Real World Example

User enters URL:

`https://localhost:5001/Books/Details/100`

Suppose Book Id 100 does not exist.

Result:

404 Not Found

Common Reasons for 404 Error

1. Invalid URL

Wrong:

`/Booksss/Index`

Correct:

`/Books/Index`

2. Controller Not Found

URL:

`/Employee/Index`

But:

EmployeeController does not exist.

Result:

404 Error

3. Action Method Not Found

URL:

`/Books/GetBooks`

But:

GetBooks() action does not exist.

Result:

404 Error

4. Resource Does Not Exist

Example:

Book Id 999

Database contains:

1,2,3,4,5

Book not found.

Return:

404 -----

----- 5. Wrong Routing Configuration

Route does not match URL.

Result:

404 Error -----

----- What is Resource?

A resource can be:

✓ Web Page

✓ Controller Action

✓ Book Record

✓ Product Record

✓ Image

✓ PDF

✓ API Endpoint -----

----- Resource Not Found

Example Controller

```
public IActionResult Details(int id)
```

```
{
```

```
var book =
```

```
_repo.GetByld(id);
```

```
if(book == null)
```

```
{  
return NotFound();  
}  
return View(book);  
}
```

User Requests

/Books/Details/100

Book does not exist.

MVC Returns

```
return NotFound();
```

Output

HTTP Status Code

404 Not Found

What Does NotFound() Do?

NotFound() returns a 404 HTTP Status Code.

Example:

```
return NotFound();
```

Equivalent To:

```
return StatusCode(404);
```

Difference Between Null and NotFound()

Bad

```
if(book == null)
```

```
{
```

```
return View();
```

```
}
```

User sees empty page.

Good

```
if(book == null)
{
return NotFound();
}
```

User sees proper 404 error.

Custom 404 Page

Program.cs

```
app.UseStatusCodePagesWithReExecute(
"/Home/Error404");
```

Controller

```
public IActionResult Error404()
{
return View();
}
```

Error404.cshtml

```
<h1>404</h1>
```

```
<p>
```

Resource Not Found

```
</p>
```

Benefits of Custom 404 Page

✓ Better User Experience

✓ Professional Look

✓ User Guidance

✓ Branding

404 vs 500 Error

404

Resource Not Found

Example:

Book does not exist.

500

Internal Server Error

Example:

Application crashed.

Database connection failed.

404 vs 400 Error

404

Resource does not exist.

Example:

/Books/999

400

Bad request from client.

Example:

Invalid input.

Real World Examples

Amazon

Product not found.

404

Netflix

Movie not found.

404

BookStore

Book Id does not exist.

404

Best Practice

Always check for null before returning View.

Example:

```
public IActionResult Details(int id)
{
    var book =
        _repo.GetByld(id);
    if(book == null)
    {
        return NotFound();
    }
    return View(book);
}
```

Interview Questions

Q. What is a 404 Error?

404 is an HTTP Status Code that indicates the requested resource could not be found. ---

Q. What does Resource Not Found mean?

The requested page, record, file, or endpoint does not exist. -----

Q. What method returns 404 in MVC?

return NotFound();

Q. Difference between 404 and 500?

404:

Resource missing.

500:

Server/Application error.

Q. Why should we use NotFound() instead of returning an empty View?

It provides the correct HTTP status code and better user experience.

One-Line Interview Answer

A 404 (Not Found) error occurs when a requested resource such as a page, controller action, file, or database record cannot be found, and in ASP.NET Core MVC it is commonly returned using the `NotFound()` method.

===== USING
ATTRIBUTE ROUTING IN ASP.NET CORE MVC

What is Attribute Routing?

Attribute Routing is a routing technique where routes are defined directly on Controllers or Action Methods using attributes.

Simple Definition:

"Instead of defining routes in `Program.cs`, we define routes directly on actions using attributes."

Why Do We Need Attribute Routing?

Conventional Routing:

`/Books/Details/5`

Works fine.

But sometimes we need custom URLs.

Examples:

`/books/all`

`/books/top-selling`

`/books/programming`

Attribute Routing makes this easy.

Syntax

```
[Route("books/all")]
```

```
public IActionResult Index()
```

```
{
```

```
    return View();
```

```
}
```

URL:

`/books/all`

Controller Level Route

```
[Route("books")]
```

```
public class BooksController : Controller
```

```
{
```

```
[Route("all")]
```

```
public IActionResult Index()
```

```
{
```

```
return View();
```

```
}
```

```
}
```

URL:

```
/books/all -----
```

----- Action Level Route

```
public class BooksController : Controller
```

```
{
```

```
[Route("books/details")]
```

```
public IActionResult Details()
```

```
{
```

```
return View();
```

```
}
```

```
}
```

URL:

```
/books/details -----
```

----- Route Parameters

```
[Route("books/details/{id}")]
```

```
public IActionResult Details(int id)
```

```
{
```

```
return View();
```

```
}
```

URL:

```
/books/details/5
```

MVC automatically sets:

id = 5

Multiple Routes

```
[Route("books")]
```

```
[Route("library")]
```

```
public IActionResult Index()
```

```
{
```

```
    return View();
```

```
}
```

Both URLs work:

/books

/library

HTTP Method + Route

```
[HttpGet("books/all")]
```

```
public IActionResult Index()
```

```
{
```

```
    return View();
```

```
}
```

Only GET requests allowed.

Advantages of Attribute Routing

✓ More Control

✓ Cleaner URLs

✓ Easy API Development

✓ SEO Friendly URLs

✓ Route Close to Action

Interview Questions

Q. What is Attribute Routing?

Attribute Routing allows routes to be defined directly on Controllers and Actions using Route attributes.

Q. Difference between Conventional Routing and Attribute Routing?

Conventional Routing:

Defined in Program.cs

Attribute Routing:

Defined using attributes.

Q. How do you define route parameters?

[Route("books/details/{id}")]

One-Line Interview Answer

Attribute Routing allows developers to define URL patterns directly on Controllers and Action Methods using attributes such as [Route], [HttpGet], and [HttpPost].

=====

UNDERSTANDING ASP.NET CORE MVC REQUEST LIFE CYCLE

What is Request Life Cycle?

Request Life Cycle is the sequence of steps that occur when a user sends a request to an ASP.NET Core MVC application.

Simple Definition:

"It describes how a request travels from Browser to Controller and back to Browser."

Real World Example

User opens:

<https://localhost:5001/Books/Details/5>

MVC processes the request and returns a page.

MVC Request Life Cycle

Browser

|

V

Kestrel Web Server

|

V

Middleware Pipeline

|

V

Routing Engine

|

V

Controller

|

V

Action Method

|

V

Model / Repository

|

V

View

|

V

HTML Response

|

V

Browser

Step 1: Browser Sends Request

User enters URL:

/Books/Details/5

Browser sends HTTP Request. -----

Step 2: Kestrel Receives Request

Kestrel is the built-in web server in ASP.NET Core.

Responsibilities:

✓ Listen for Requests

✓ Process Requests

✓ Forward Requests -----

Step 3: Middleware Pipeline Executes

Middleware examples:

UseHttpsRedirection()

UseStaticFiles()

UseAuthentication()

UseAuthorization()

Request passes through middleware one by one.

Step 4: Routing Engine Executes

Routing Engine analyzes URL.

Example:

/Books/Details/5

Finds:

Controller = Books

Action = Details

Id = 5

----- Step 5: Controller Created

MVC creates:

BooksController

Using Dependency Injection.

----- Step 6: Action Method Executes

MVC calls:

public IActionResult Details(int id)

{
}

Parameter:

id = 5

----- Step 7: Model / Repository Access

Controller requests data.

Example:

```
var book =  
_repo.GetById(5);
```

Step 8: Action Returns Result

Example:

```
return View(book);
```

MVC receives ViewResult.

Step 9: View Engine Executes

MVC locates:

Views/Books/Details.cshtml

View is rendered.

Step 10: HTML Generated

Razor View converts:

HTML + C#

into pure HTML.

Step 11: Response Sent

HTML returned to browser.

Step 12: Browser Displays Page

User sees:

Book Details Page

Life Cycle Diagram

User Request

I

V

Kestrel

I

V

Middleware

|

V

Routing

|

V

Controller

|

V

Action

|

V

Model/Repository

|

V

View

|

V

HTML Response

|

V

Browser

Important Components in Life Cycle

1. Kestrel

Web Server

2. Middleware

Processes Requests

3. Routing

Finds Controller & Action

4. Controller

Handles Request

5. Action Method

Executes Logic

6. Model/Repository

Provides Data

7. Razor View

Generates HTML

Interview Questions

Q. What is Request Life Cycle?

The sequence of steps that occur when a request is processed by ASP.NET Core MVC.

Q. What is the first component that receives the request?

Kestrel Web Server.

Q. What is the role of Middleware?

Processes requests and responses before reaching MVC.

Q. What is the role of Routing?

Maps URL to Controller and Action.

Q. What is the role of Razor View?

Generates HTML output.

One-Line Interview Answer

The ASP.NET Core MVC Request Life Cycle starts when a browser sends a request, passes through Kestrel, Middleware, Routing, Controller, Action Method, and View Engine, and finally

returns an HTML response back to the browser.

===== LAYOUT AND USING WITH ASSOCIATED VIEWS IN ASP.NET CORE MVC

What is a Layout?

A Layout is a master page that provides a common structure for multiple views.

Simple Definition:

"Layout is a common template shared by multiple pages."

Why Do We Need Layout?

Without Layout

Every page contains:

- ✓ Header
- ✓ Navigation Menu
- ✓ Footer
- ✓ CSS References

Repeated again and again.

Problems:

- Duplicate Code
 - Difficult Maintenance
 - Inconsistent UI
-

With Layout

Header

Navigation

Footer

Created Once

Used Everywhere

Real World Example

BookStore Website

Pages:

Home

Books

Authors

Categories

Contact

All pages use same:

Header

Menu

Footer

Layout solves this problem.

Layout Structure

Header

Navigation Menu

@RenderBody()

Footer

Layout File Location

Views

|

+-- Shared

|

+-- _Layout.cshtml

MVC automatically searches here.

Example Layout

_Layout.cshtml

<!DOCTYPE html>

<html>

<head>

```
<title>BookStore</title>
</head>
<body>
<header>
<h1>BookStore</h1>
</header>
<nav>
<a href="/">Home</a>
<a href="/Books">Books</a>
</nav>
<hr />
@RenderBody()
<hr />
<footer>
Copyright 2026
</footer>
</body>
</html>
```

What is RenderBody()?

RenderBody() is a placeholder where the content of the current View is displayed.

Example:

```
@RenderBody()
```

Think of it as:

"Insert page content here."

Associated View Example

Index.cshtml

```
<h2>Book List</h2>
<p>All books available here.</p>
```

Output

Header

Navigation

Book List

All books available here.

Footer

How Layout Works?

Request:

/Books/Index

MVC Loads:

Index.cshtml

MVC Finds Layout

_Layout.cshtml

MVC Replaces

@RenderBody()

with

Index.cshtml Content

Final HTML Sent To Browser

Header

Navigation

Book List

Footer

How to Connect Views with Layout?

Method 1 (Recommended)

Using _ViewStart.cshtml

_ViewStart.cshtml

Location:

Views

|

+-- _ViewStart.cshtml -----

----- Code

```
@{  
Layout = "_Layout";  
}
```

Purpose

Automatically applies layout to all views.

No need to specify layout in every page. -

Method 2

Inside Individual View

Index.cshtml

```
@{  
Layout = "_Layout";  
}
```

Not Recommended

Because every page must repeat it. -----

----- What is

_ViewStart.cshtml?

Special file executed before every View.

Used to:

✓ Set Layout

✓ Common View Configuration -----

----- Project

Structure

Views

|

+-- Shared

| |

| +-- _Layout.cshtml

|

+-- Books

| |

| +-- Index.cshtml

| +-- Create.cshtml

|

+-- _ViewStart.cshtml

Using Multiple Layouts

Example:

Admin Layout

_AdminLayout.cshtml

Customer Layout

_Layout.cshtml

Controller

```
public IActionResult Dashboard()
```

```
{
```

```
    return View();
```

```
}
```

View

```
@{
```

```
    Layout = "_AdminLayout";
```

```
}
```

Output uses Admin Layout.

What is RenderSection()?

Used for optional sections.

Layout

```
@RenderSection(
```

```
"Scripts",
```

```
required:false)
```

View

```
@section Scripts
```

```
{
```

```
<script>
```

```
alert("Hello");
```

```
</script>
```

```
}
```

Purpose

Allows page-specific content.

Benefits of Layout

✓ Reusable UI

✓ Consistent Design

✓ Easy Maintenance

✓ Cleaner Views

✓ Less Code Duplication

Without Layout

Index.cshtml

Header

Footer

Create.cshtml

Header

Footer

Edit.cshtml

Header

Footer

Repeated everywhere.

With Layout

Header/Footer written once.

All pages automatically use them.

Real World Example

Amazon

Common Header

Common Navigation

Common Footer

Netflix

Common Menu

Common Footer

BookStore

Common Menu

Common Branding

Common Footer

Interview Questions

Q. What is a Layout in MVC?

A Layout is a master page that provides a common structure for multiple views.

Q. What is the purpose of `RenderBody()`?

`RenderBody()` acts as a placeholder where the content of the current View is rendered.

Q. Where is Layout file usually stored?

Views/Shared/_Layout.cshtml

Q. What is _ViewStart.cshtml?

A special file that sets the default Layout for all views.

Q. Difference between Layout and Partial View?

Layout:

Entire page template.

Partial View:

Reusable UI component.

Q. What is RenderSection()?

Used to define optional sections such as page-specific scripts or styles.

Q. What are Associated Views?

Associated Views are views that use a Layout and render their content inside the Layout's RenderBody() section.

One-Line Interview Answer

A Layout is a master page (_Layout.cshtml) that provides a common UI structure for multiple views, while associated views render their specific content inside the Layout using the RenderBody() placeholder. =====

BUNDLING AND MINIFICATION IN ASP.NET CORE MVC

What is Bundling?

Bundling is the process of combining multiple CSS or JavaScript files into a single file.

Simple Definition:

"Many files become one file."

Example

Without Bundling

site.css

bootstrap.css

menu.css

theme.css

Total Requests = 4

With Bundling

bundle.css

Total Requests = 1

Why Do We Need Bundling?

Every CSS or JavaScript file requires a separate HTTP request.

More Files

↓

More Requests

↓

Slower Website

Bundling Reduces

✓ Number of Requests

✓ Network Traffic

✓ Page Load Time

✓ Server Load

Real World Example

Without Bundling

Browser Requests:

bootstrap.css

site.css

theme.css

menu.css

jquery.js

custom.js

Total Requests = 6

With Bundling

bundle.css

bundle.js

Total Requests = 2

What is Minification?

Minification is the process of removing unnecessary characters from CSS and JavaScript files.

Simple Definition:

"Make files smaller by removing spaces, comments, and unnecessary formatting."

Before Minification

```
function Add()  
{  
  var a = 10;  
  var b = 20;  
  return a + b;  
}
```

After Minification

```
function Add(){var a=10;var b=20;return a+b;}
```

Output is Same

File Size is Smaller

What Gets Removed?

✓ Spaces

✓ Tabs

✓ New Lines

✓ Comments

✓ Unnecessary Characters

CSS Example

Before

```
body
{
background-color: white;
} -----
```

- After

```
body{background-color:white;} -----
-----
```

Benefits of Minification

- ✓ Smaller File Size
- ✓ Faster Download
- ✓ Faster Website
- ✓ Better Performance -----

----- Bundling +

Minification

Multiple Files

|

V

Bundle Together

|

V

Remove Spaces/Comments

|

V

Smaller Optimized File -----

----- Real World

Example

Before Optimization

bootstrap.css = 200 KB

site.css = 100 KB

theme.css = 150 KB

Total = 450 KB -----

After Bundling & Minification

bundle.min.css = 250 KB

Smaller and Faster

Why Important in Production?

Development Environment

Readable Code

Easy Debugging

Production Environment

Optimized Files

Faster Performance

Better User Experience

Common File Naming Convention

site.css

Minified Version

site.min.css

app.js

Minified Version

app.min.js

ASP.NET Core Approach

Older ASP.NET MVC (.NET Framework)

Used:

BundleConfig.cs

ASP.NET Core

Typically uses:

Webpack

Gulp

Vite

Build Tools

Or Pre-Minified Files

Example

Development

```
<link href="site.css"
rel="stylesheet" />
```

Production

```
<link href="site.min.css"
rel="stylesheet" />
```

Benefits of Bundling and Minification

- ✓ Faster Page Load
 - ✓ Reduced Bandwidth
 - ✓ Better Performance
 - ✓ Improved User Experience
 - ✓ Better SEO Rankings
-

Real World Companies

Amazon

Netflix

Google

Microsoft

All use Bundling and Minification.

Interview Questions

Q. What is Bundling?

Bundling is the process of combining multiple CSS and JavaScript files into a single file to reduce HTTP requests.

Q. What is Minification?

Minification is the process of removing unnecessary characters such as spaces, comments, and line breaks to reduce file size.

Q. Why do we use Bundling?

To reduce the number of HTTP requests and improve application performance.

Q. Why do we use Minification?

To reduce file size and improve page loading speed.

Q. What is the benefit of Bundling and Minification?

- Faster Website
- Reduced Network Traffic
- Better Performance
- Better User Experience

Q. What is a Minified File?

A compressed version of CSS or JavaScript.

Example:

site.min.css

app.min.js

Q. Is Bundling and Minification mainly used in Development or Production?

Production.

One-Line Interview Answer

Bundling combines multiple CSS and JavaScript files into a single file, while Minification removes unnecessary characters to reduce file size, improving application performance and page load speed. =====

ERROR HANDLING IN ASP.NET CORE MVC WITH LOG ENTRY

What is Error Handling?

Error Handling is the process of identifying, managing, and responding to application errors gracefully.

Simple Definition:

"Error Handling prevents application crashes and provides meaningful responses to users."

Why Do We Need Error Handling?

Without Error Handling

Application Crashes

User Sees Technical Error

Poor User Experience

With Error Handling

✓ Friendly Error Page

✓ Log Error Details

✓ Application Continues Running

✓ Easier Troubleshooting

Real World Example

BookStore Application

User Requests:

/Books/Details/100

Database Connection Fails.

Without Error Handling

Application Crash

With Error Handling

Error Logged

Friendly Error Page Displayed

What is Logging?

Logging means recording application events, warnings, and errors into a file, database, or monitoring system.

Simple Definition:

"Logging is writing application information for future analysis."

Why Logging is Important?

When an error occurs:

Developer wants to know:

✓ What happened?

✓ When did it happen?

✓ Which user was affected?

✓ Which page failed?

Logging provides answers. -----

----- ILogger in ASP.NET

Core

ASP.NET Core provides built-in logging.

Example

```
private readonly ILogger<BooksController>
```

```
_logger; -----
```

---- Constructor Injection

```
public class BooksController
```

```
: Controller
```

```
{
```

```
private readonly ILogger<BooksController>
```

```
_logger;
```

```
public BooksController(
```

```
ILogger<BooksController> logger)
```

```
{
```

```
_logger = logger;
```

```
}
```

```
}
```

```
Logging Information
```

```
_logger.LogInformation(
```

```
"Books page opened"); -----
```

----- Logging Warning

```
_logger.LogWarning(  
    "Book not found");
```

Logging Error

```
_logger.LogError(  
    "Database connection failed");
```

Try-Catch with Logging

```
public IActionResult Details(int id)  
{  
    try  
    {  
        var book =  
            _repo.GetByld(id);  
        return View(book);  
    }  
    catch(Exception ex)  
    {  
        _logger.LogError(  
            ex,  
            "Error while loading book");  
        return View("Error");  
    }  
}
```

What Gets Logged?

Date

Time

Error Message

Stack Trace

User Information

Controller Name

Action Name -----

----- Example Log Entry

[2026-06-18 10:30]

ERROR

BooksController

Details()

Book Id = 10

Database Connection Failed -----

----- Global

Error Handling

Program.cs

app.UseExceptionHandler(

"/Home/Error"); -----

----- Controller

public IActionResult Error()

{

return View();

} -----

- Error View

<h1>

Something went wrong

</h1> -----

----- Benefits

✓ Centralized Error Handling

✓ Cleaner Controllers

✓ Better Maintenance -----

----- 404 Error

Handling

public IActionResult Details(int id)

```
{  
var book =  
_repo.GetByld(id);  
if(book == null)  
{  
_logger.LogWarning(  
$"Book {id} not found");  
return NotFound();  
}  
return View(book);  
}
```

Best Practices

- ✓ Always Log Exceptions
 - ✓ Show Friendly Error Messages
 - ✓ Never Show Stack Trace To Users
 - ✓ Use Global Error Handling
 - ✓ Log Critical Information
-

Interview Questions

Q. What is Error Handling?

Error Handling is the process of managing application errors gracefully.

Q. What is Logging?

Logging is recording application events and errors for monitoring and troubleshooting.

Q. Which service is used for logging?

ILogger<T>

Q. Why should we log exceptions?

To diagnose and troubleshoot issues.

One-Line Interview Answer

Error Handling manages application failures gracefully, while Logging records errors and application events using ILogger for monitoring and troubleshooting.

=====

FILTERS AND CUSTOM ACTION FILTERS IN ASP.NET CORE MVC

What are Filters?

Filters are components that execute before or after an Action Method.

Simple Definition:

"Filters allow us to run common code before or after controller actions."

Why Do We Need Filters?

Suppose every action requires:

✓ Logging

✓ Authentication

✓ Auditing

✓ Performance Tracking

Writing the same code repeatedly is bad.

Filters solve this problem.

Without Filters

```
public IActionResult Index()
{
    Log();
    // Action Code
}

public IActionResult Create()
{
    Log();
    // Action Code
}
```

Repeated code everywhere.

With Filters

Filter handles logging automatically.

Cleaner code.

MVC Filter Execution Flow

Request

I

V

Filter

I

V

Action Method

I

V

Filter

I

V

Response -----

----- Types of Filters

1. Authorization Filter

2. Action Filter

3. Result Filter

4. Exception Filter -----

1. Authorization Filter

Runs first.

Used for:

✓ Authentication

✓ Authorization

Example

[Authorize] -----

2. Action Filter

Runs before and after action execution.

Most commonly used.

3. Result Filter

Runs before and after View rendering. -----

4. Exception Filter

Handles exceptions globally. -----

What is Action Filter?

Action Filter executes before and after an Action Method. -

Creating Custom Action Filter

using Microsoft.AspNetCore.Mvc.Filters;

public class LoggingFilter

: ActionFilterAttribute

{

}

Before Action Execution

public override void

OnActionExecuting(

ActionExecutingContext context)

{

Console.WriteLine(

"Before Action");

}

After Action Execution

public override void

OnActionExecuted(

ActionExecutedContext context)

{

Console.WriteLine(

```
"After Action");
```

```
} -----
```

- Complete Example

```
public class LoggingFilter
```

```
: ActionFilterAttribute
```

```
{
```

```
public          override          void
```

```
OnActionExecuting(
```

```
ActionExecutingContext context)
```

```
{
```

```
Console.WriteLine(
```

```
"Action Started");
```

```
}
```

```
public          override          void
```

```
OnActionExecuted(
```

```
ActionExecutedContext context)
```

```
{
```

```
Console.WriteLine(
```

```
"Action Completed");
```

```
}
```

```
} -----
```

- Applying Filter

```
[LoggingFilter]
```

```
public IActionResult Index()
```

```
{
```

```
return View();
```

```
} -----
```

- Execution Sequence

```
Request
```

```
↓
```

Action Started
↓

Index()

↓

Action Completed

↓

Response -----

----- Real World Uses of Filters

✓ Logging

✓ Auditing

✓ Security Checks

✓ Performance Monitoring

✓ User Activity Tracking -----

----- Audit Filter

Example

```
public class AuditFilter
```

```
: ActionFilterAttribute
```

```
{
```

```
public override void
```

```
OnActionExecuting(
```

```
ActionExecutingContext context)
```

```
{
```

```
Console.WriteLine(
```

```
"User Access Logged");
```

```
}
```

```
} -----
```

```
- Global Filter Registration
```

```
builder.Services
```

```
.AddControllersWithViews(
```

```
options =>
```

```
{
options.Filters
.Add<LoggingFilter>();
});
```

Applied to all controllers.

Advantages of Filters

- ✓ Reusable Code
 - ✓ Cleaner Controllers
 - ✓ Centralized Logic
 - ✓ Better Maintainability
-

Interview Questions

Q. What is a Filter?

A Filter is a component that executes before or after an MVC action. -----

Q. What is an Action Filter?

A Filter that runs before and after Action Method execution.

Q. What are the main types of Filters?

Authorization

Action

Result

Exception

Q. Which method runs before an action?

OnActionExecuting()

Q. Which method runs after an action?

OnActionExecuted()

Q. Why use Custom Action Filters?

To implement reusable cross-cutting concerns like logging, auditing, and performance monitoring.

One-Line Interview Answer

Filters are components that execute before or after MVC actions, and Custom Action Filters are commonly used to implement reusable functionality such as logging, auditing, authorization, and performance monitoring.

===== AUTHORIZE &
ALLOWANONYMOUS ATTRIBUTES IN ASP.NET CORE MVC

Why Do We Need Security?

Not every page should be accessible to everyone.

Examples:

Public Pages

✓ Home Page

✓ About Us

✓ Contact Us

Protected Pages

✓ Admin Dashboard

✓ Employee Management

✓ User Profile

✓ Orders

These pages require login.

What is [Authorize]?

[Authorize] is an attribute that restricts access to authenticated users only.

Simple Definition:

"Only logged-in users can access this page."

Example

[Authorize]

```
public IActionResult Dashboard()
```

```
{
```

```
    return View();
```

```
}
```

What Happens?

User Logged In

✓ Access Granted -----

----- User Not Logged In

■ Access Denied

Redirected To Login Page -----

----- Real World

Example

Internet Banking

Account Details

Fund Transfer

Transaction History

Only logged-in users can access.

Implemented using:

[Authorize] -----

----- Controller Level

Authorization [Authorize]

public class AdminController

: Controller

{

public IActionResult Dashboard()

{

return View();

}

public IActionResult Users()

{

return View();

}

} -----

-

Result

Entire Controller Protected

Dashboard()

Users()

All require login. -----

----- Action Level

Authorization

public class HomeController

: Controller

{

[Authorize]

public IActionResult Dashboard()

{

return View();

}

public IActionResult Index()

{

return View();

}

} -----

- Result

Dashboard

Requires Login -----

----- Index

Accessible To Everyone -----

----- Role-Based

Authorization

Only Admin Users

[Authorize(Roles = "Admin")]

public IActionResult Dashboard()

```
{  
return View();  
}
```

Only Users with Admin Role
Can Access Dashboard

Multiple Roles
[Authorize(
Roles = "Admin,Manager")]
public IActionResult Reports()
{
return View();
}

Allowed Roles
Admin
Manager

What is [AllowAnonymous]?
[AllowAnonymous] allows access without authentication.
Simple Definition:
"Anyone can access this page, even without login."

Example
[AllowAnonymous]
public IActionResult Login()
{
return View();
}

Why Use AllowAnonymous?
Login Page

Registration Page

Forgot Password

Public Information Pages

Should be accessible without login. ---

Real World Example

Bank Website

Home Page

Login Page

Register Page

Need:

[AllowAnonymous] -----

----- Controller Level

Example

[Authorize]

public class AccountController

: Controller

{

[AllowAnonymous]

public IActionResult Login()

{

return View();

}

public IActionResult Profile()

{

return View();

}

} -----

- Result

Login()

Accessible To Everyone

Profile()

Requires Login

Authorize vs AllowAnonymous

AUTHORIZE

Requires Authentication

ALLOWANONYMOUS

No Authentication Required

Example

[Authorize]

public IActionResult Orders()

{

return View();

}

Login Required

[AllowAnonymous]

public IActionResult ContactUs()

{

return View();

}

No Login Required

How Authorization Works?

User Request

|

V

Authorization Middleware

|

V

Check User Login

|

+---- No

||

| V

| Redirect Login

|

+---- Yes

|

V

Execute Action

Authentication vs Authorization

Authentication

Who are you?

Example:

Login Verification

Authorization

What can you access?

Example:

Admin Access

Memory Trick

Authentication

Identity Check

Authorization

Permission Check

BookStore Example

Public Pages

[AllowAnonymous]

Home

Book Catalog

Contact

Protected Pages

[Authorize]

Add Book

Edit Book

Delete Book

Admin Dashboard

Common Usage

Home Page

[AllowAnonymous]

Login Page

[AllowAnonymous]

Register Page

[AllowAnonymous]

Dashboard

[Authorize]

Admin Panel

[Authorize(Roles="Admin")]

Interview Questions

Q. What is Authorize Attribute?

Authorize attribute restricts access to authenticated users.

Q. What is AllowAnonymous Attribute?

AllowAnonymous allows access without authentication.

Q. Difference Between Authorize and AllowAnonymous?

Authorize:

Login Required.

AllowAnonymous:

Login Not Required.

Q. Can Authorize be applied at Controller level?

Yes.

It protects all actions inside the controller.

Q. Can AllowAnonymous override Authorize?

Yes.

AllowAnonymous overrides Authorize for a specific action.

Q. What is Role-Based Authorization?

Restricting access based on user roles.

Example:

[Authorize(Roles="Admin")]

Q. Difference Between Authentication and Authorization?

Authentication:

Who are you?

Authorization:

What can you access?

One-Line Interview Answer

[Authorize] restricts access to authenticated or authorized users, while [AllowAnonymous] allows anyone to access an action or controller without authentication, even when authorization is applied elsewhere. =====

PREVENTING FORGERY ATTACK USING ANTIFORGERYTOKEN

What is a Forgery Attack?

A Forgery Attack (CSRF - Cross Site Request Forgery) occurs when a malicious website tricks a logged-in user into performing unwanted actions on another website.

Simple Definition:

"A hacker forces a logged-in user to submit a request without their knowledge."

Real World Example

User logs into BookStore Website.

User keeps session active.

Meanwhile

User visits Hacker Website.

Hacker Website automatically submits:

Delete Book

Transfer Money

Change Password

using User's Session.

Result

Server thinks:

"Request came from valid user."

Dangerous!

What is CSRF?

CSRF

Cross Site Request Forgery

Meaning

Cross Site

Request originates from another website.

Forgery

Request is fake.

Attack Flow

User Logs In

I

V

Authentication Cookie Created

I

V

User Visits Malicious Website

I

V

Fake Form Submitted

I

V

Cookie Sent Automatically

I

V

Server Executes Request

How ASP.NET Core Prevents CSRF?

Using:

Anti-Forgery Token

What is Anti-Forgery Token?

A unique hidden security token generated by ASP.NET Core.

Simple Definition:

"A secret token that proves the request originated from the actual website."

How It Works?

Form Generated

I

V

Token Created

I

V

Token Sent To Browser

I

V

Form Submitted

I

V

Server Validates Token

I

V

Request Accepted

Adding Anti-Forgery Token

View

<form method="post">

@Html.AntiForgeryToken()

</form>

Generated HTML

<input

type="hidden"

name="__RequestVerificationToken"

value="ABC123XYZ..." />

Controller Validation

[HttpPost]

[ValidateAntiForgeryToken]

public IActionResult Create(Book book)

{

return RedirectToAction("Index");

}

What Happens?

Token Valid

✓ Request Accepted

Token Missing

■ Request Rejected

Token Modified

■ Request Rejected

BookStore Example

Create Book Form

```
<form method="post">
@Html.AntiForgeryToken()
<input asp-for="Title" />
<button type="submit">
Save
</button>
</form>
```

Controller

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Book book)
{
    _repo.Add(book);
    return RedirectToAction("Index");
}
```

Best Practice

Always Use:

@Html.AntiForgeryToken()

AND

[ValidateAntiForgeryToken]

For:

Create

Edit

Delete

Any POST Action

Interview Questions

Q. What is CSRF?

Cross Site Request Forgery is an attack where a malicious site tricks a user into submitting unwanted requests.

Q. How does ASP.NET Core prevent CSRF?

Using Anti-Forgery Tokens.

Q. Which attribute validates Anti-Forgery Token?

[ValidateAntiForgeryToken]

Q. Which helper generates Anti-Forgery Token?

@Html.AntiForgeryToken()

One-Line Interview Answer

ASP.NET Core prevents CSRF attacks by generating a unique Anti-Forgery Token using @Html.AntiForgeryToken() and validating it using [ValidateAntiForgeryToken].

=====

PREVENTING CROSS SITE SCRIPTING (XSS) ATTACK

What is XSS?

XSS stands for:

Cross Site Scripting

Simple Definition

"An attacker injects malicious JavaScript into a web page."

Real World Example

Suppose a user enters:

```
<script>
```

```
alert('Hacked')
```

```
</script>
```

into a comment box.

Without Protection

Comment gets stored.

Every visitor executes:

```
alert('Hacked')
```

Dangerous Effects

✓ Steal Cookies

✓ Steal Session Data

✓ Redirect Users

✓ Modify Page Content

✓ Capture Passwords

Attack Flow

Attacker Enters Script

|

V

Script Saved

|

V

Other Users Open Page

|

V

Script Executes In Browser

Example Attack

Comment:

```
<script>
```

```
document.location=  
'https://hacker.com'  
</script>
```

User Opens Page

Browser executes script.

How ASP.NET Core Prevents XSS?

By Default:

Razor automatically encodes output.

Example

User Input

```
<script>  
alert('Hacked')  
</script>
```

View

@Model.Comment

Output

```
&lt;script&gt;  
alert('Hacked')  
&lt;/script&gt;
```

Result

Displayed as text.

Script NOT executed.

HTML Encoding

Special Characters Converted

< becomes <

> becomes >

This is called:

HTML Encoding

Safe Example

@Model.Title

Recommended.

Dangerous Example

@Html.Raw(Model.Title)

Why Dangerous?

Html.Raw()

Disables encoding.

Browser executes script.

Example

User Input

<script>

alert('Hacked')

</script>

View

@Html.Raw(Model.Comment)

Result

JavaScript Executes

XSS Attack Successful

Best Practices To Prevent XSS

✓ Use Razor Syntax

@Model.Title

✓ Avoid `Html.Raw()`

Unless absolutely necessary.

✓ Validate User Input

Use Data Annotations.

✓ Sanitize HTML Content

Remove dangerous tags.

✓ Use Content Security Policy (CSP)

Extra browser protection.

BookStore Example

User Review

`@Model.Review`

Safe

`@Html.Raw(Model.Review)`

Unsafe

CSRF vs XSS

CSRF

Tricks user into sending requests.

XSS

Injects malicious scripts into web pages.

Interview Questions

Q. What is XSS?

Cross Site Scripting is an attack where malicious JavaScript is injected into a web page.

Q. How does ASP.NET Core prevent XSS?

By automatically HTML encoding Razor output.

Q. Which syntax is safe?

@Model.Title

Q. Which syntax is dangerous?

@Html.Raw()

Q. What is HTML Encoding?

Converting special characters into safe HTML entities.

Q. Difference Between CSRF and XSS?

CSRF:

Forces unwanted requests.

XSS:

Injects malicious JavaScript.

One-Line Interview Answer

ASP.NET Core prevents XSS attacks by automatically HTML encoding Razor output, ensuring that user-entered scripts are displayed as text rather than executed by the browser.

ENTITY FRAMEWORK CORE (EF CORE)

What is EF Core?

Entity Framework Core (EF Core) is Microsoft's Object Relational Mapper (ORM) for .NET applications.

Simple Definition:

"EF Core allows us to work with a database using C# objects instead of writing SQL queries."

Why Do We Need EF Core?

Without EF Core

Write SQL Manually

Open Connection

Execute Query

Read Data

Convert Data To Objects

Example

```
SELECT * FROM Books
```

Then manually read records.

Lots of code.

With EF Core

```
var books =
```

```
_dbContext.Books.ToList();
```

Simple and clean.

What is ORM?

ORM = Object Relational Mapper

Object

C# Class

Relational

Database Table

Mapper

Converts between them.

Example

Database Table

Books

Id

Title

Author

Price

C# Class

public class Book

{

public int Id { get; set; }

public string Title { get; set; }

public string Author { get; set; }

public decimal Price { get; set; }

}

EF Core maps:

Book Class ↔ Books Table

How EF Core Works?

Controller

|

V

Repository

|

V

DbContext

|

V

EF Core

|

V

Database

What is DbContext?

DbContext is the main class in EF Core that manages database operations.

Simple Definition:

"DbContext is a bridge between application and database."

Example

```
public class ApplicationDbContext
: DbContext
{
    public DbSet<Book> Books
    {
        get;
        set;
    }
}
```

What is DbSet?

DbSet represents a database table.

Example

```
public DbSet<Book> Books
{
    get;
    set;
}
```

Represents:

Books Table

CRUD Operations Using EF Core

CREATE

Read

Update

Delete

INSERT

```
var book = new Book
{
    Title = "Clean Code",
    Author = "Robert Martin",
    Price = 999
};
_dbContext.Books.Add(book);
_dbContext.SaveChanges();
```

SELECT

```
var books =
_dbContext.Books.ToList();
```

SELECT BY ID

```
var book =
_dbContext.Books
.Find(1);
```

UPDATE

```
var book =
_dbContext.Books.Find(1);
book.Price = 1200;
_dbContext.SaveChanges();
```

DELETE

```
var book =  
_dbContext.Books.Find(1);  
_dbContext.Books.Remove(book);  
_dbContext.SaveChanges(); -----  
-----
```

What is SaveChanges()?

SaveChanges() commits changes to the database.

Example

```
_dbContext.SaveChanges();
```

Without SaveChanges()

Database is not updated. -----

How to Configure EF Core?

Program.cs

```
builder.Services.AddDbContext<  
ApplicationDbContext>(  
options =>  
options.UseSqlServer(  
connectionString)); -----  
-----
```

Connection String

appsettings.json

```
{  
  "ConnectionStrings":  
  {  
    "DefaultConnection":  
    "Server=.;Database=BookStoreDb;  
Trusted_Connection=True;"  
  }  
}
```

Advantages of EF Core

- ✓ Less SQL Code
- ✓ Faster Development
- ✓ Strongly Typed
- ✓ LINQ Support
- ✓ Database Independence
- ✓ Easy CRUD Operations

What is LINQ?

Language Integrated Query

Used to query database using C# syntax.

Example

```
var books =  
_dbContext.Books  
.Where(x => x.Price > 500)  
.ToList();
```

EF Core Approaches

1. Database First

Database Exists First

Generate Models From Database

2. Code First

Create Models First

Generate Database Automatically

Most Popular

Code First Example

```
public class Book  
{  
    public int Id { get; set; }  
    public string Title  
{  
        get;
```

set;

}

}

EF Core creates table automatically. -----

Migration

Migration keeps database schema synchronized with Models. -

Add Migration

Add-Migration InitialCreate -----

Update Database

Update-Database

Migration Flow

Model Created

|

V

Migration Created

|

V

Database Updated

Popular EF Core Methods

Add()

AddRange()

Find()

FirstOrDefault()

Where()

Update()

Remove()

SaveChanges()

Async Methods

SaveChangesAsync()

ToListAsync()

FindAsync()

FirstOrDefaultAsync()

Real World Example

BookStore

Book Model

Books Table

Employee Management

Employee Model

Employees Table

E-Commerce

Product Model

Products Table

Interview Questions

Q. What is EF Core?

Entity Framework Core is Microsoft's ORM framework used to interact with databases using C# objects.

Q. What is ORM?

Object Relational Mapper.

Maps C# Classes to Database Tables.

Q. What is DbContext?

Main EF Core class used to communicate with the database.

Q. What is DbSet?

Represents a database table.

Q. What is SaveChanges()?

Saves all pending changes to the database.

Q. What is LINQ?

Language Integrated Query used to query collections and databases using C# syntax. -----

Q. Difference Between Database First and Code First?

Database First:

Database already exists.

Code First:

Models created first, database generated later.

Q. What are Migrations?

A feature used to keep database schema synchronized with model changes. -----

One-Line Interview Answer

Entity Framework Core (EF Core) is Microsoft's ORM framework that allows developers to perform database operations using C# classes and LINQ instead of writing raw SQL queries.

===== USING
VALIDATION, PRIMARY KEY, FOREIGN KEY, ETC. IN ENTITY FRAMEWORK CORE

Why Do We Use Data Annotations in EF Core?

Data Annotations help us:

- ✓ Validate User Input
- ✓ Define Primary Keys
- ✓ Define Foreign Keys
- ✓ Configure Database Tables
- ✓ Control Column Behavior

Sample BookStore Database

Category

|

|

| One Category

|

V

Books

Example:

Programming Category

|

+--- Clean Code

+--- C# in Depth

+--- Design Patterns

STEP 1: CATEGORY MODEL

```
public class Category
```

```
{
```

```
[Key]
```

```
public int CategoryId
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
[Required]
```

```
[StringLength(100)]
```

```
public string CategoryName
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
public ICollection<Book> Books
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
}
```

STEP 2: BOOK MODEL

```
public class Book
```

```
{
```

```
[Key]
```

```
public int BookId
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
[Required]
```

```
[StringLength(100)]
```

```
public string Title
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
[Required]
```

```
[StringLength(50)]
```

```
public string Author
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
[Range(1,10000)]
```

```
public decimal Price
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
[Display(Name="Book Description")]
```

```

[MaxLength(500)]
public string Description
{
    get;
    set;
}
// Foreign Key
public int CategoryId
{
    get;
    set;
}
[ForeignKey("CategoryId")]
public Category Category
{
    get;
    set;
}
}

```

PRIMARY KEY

What is Primary Key?

A column that uniquely identifies each record.

Example:

BookId

```

[Key]
public int BookId
{
    get;
    set;
}

```

Database

BookId

1

2

3

4

Every value is unique. -----

FOREIGN KEY

What is Foreign Key?

A column used to create relationship between tables. -

Category Table

CategoryId

1 Programming

2 Database

3 AI

Books Table

BookId

Title

CategoryId

1 Clean Code 1

2 Oracle SQL 2

3 AI Basics 3

CategoryId is Foreign Key. -----

Foreign Key Definition

public int CategoryId

{

```
get;
set;
}
[ForeignKey("CategoryId")]
public Category Category
{
    get;
    set;
}
```

VALIDATION ATTRIBUTES

1. Required

Field cannot be empty.

```
[Required]
public string Title
{
    get;
    set;
}
```

Validation Error

Title is required.

2. StringLength

```
[StringLength(100)]
public string Title
{
    get;
    set;
}
```

Maximum 100 characters.

3. Range

[Range(1,10000)]

public decimal Price

{

get;

set;

}

Valid:

100

500

Invalid:

0

-100

4. MaxLength

[MaxLength(500)]

public string Description

{

get;

set;

}

Database:

nvarchar(500)

5. Display

[Display(Name="Book Name")]

public string Title

{

get;

set;

}

User sees:

Book Name

instead of

Title

6. EmailAddress

[EmailAddress]

public string Email

{

get;

set;

}

Validates Email Format.

7. Phone

[Phone]

public string Mobile

{

get;

set;

}

Validates Phone Number.

8. Compare

[Compare("Password")]

public string ConfirmPassword

{

get;

set;

}

Used in Registration Forms.

9. NotMapped

[NotMapped]

```
public string ConfirmPassword
```

```
{
```

```
get;
```

```
set;
```

```
}
```

Not created in database.

10. Column

```
[Column("Book_Name")]
```

```
public string Title
```

```
{
```

```
get;
```

```
set;
```

```
}
```

Database Column Name:

```
Book_Name
```

DBCONTEXT

```
public class ApplicationDbContext
```

```
: DbContext
```

```
{
```

```
public DbSet<Book> Books
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
public DbSet<Category> Categories
```

```
{
```

```
get;
```

```
set;
```

```
}
```

```
}
```

GENERATED TABLES

Categories

CategoryId (PK)

CategoryName

Books

BookId (PK)

Title

Author

Price

Description

CategoryId (FK) -----

RELATIONSHIP

Category

|

|

| One

|

V

Many Books

One-To-Many Relationship -----

CONTROLLER VALIDATION

[HttpPost]

public IActionResult Create(Book book)

{

if(ModelState.IsValid)

{

```
_context.Books.Add(book);  
_context.SaveChanges();  
return RedirectToAction(  
    "Index");  
}  
return View(book);  
}
```

What ModelState.IsValid Checks?

- ✓ Required
- ✓ Range
- ✓ StringLength
- ✓ Email
- ✓ Phone
- ✓ Compare

All Data Annotation Rules

INTERVIEW QUESTIONS

Q. What is Primary Key?

A column that uniquely identifies each row in a table.

Example:

BookId

Q. What is Foreign Key?

A column used to establish relationship between two tables.

Example:

CategoryId

Q. What is [Key]?

Used to define Primary Key.

Q. What is [ForeignKey]?

Used to define relationship between entities.

Q. What is [Required]?

Makes a field mandatory.

Q. What is [Range]?

Validates numeric values within a specified range.

Q. What is [NotMapped]?

Prevents a property from being created in database.

Q. What does ModelState.IsValid do?

Checks whether all validation rules are satisfied.

One-Line Interview Answer

EF Core Data Annotations such as [Key], [ForeignKey], [Required], [Range], [StringLength], and [NotMapped] are used to define database relationships, validation rules, and schema configuration directly within model classes.

===== FLUENT
API IN ENTITY FRAMEWORK CORE

What is Fluent API?

Fluent API is a way to configure Models, Relationships, Keys, Constraints, and Database behavior using C# code instead of Data Annotations.

Simple Definition:

"Fluent API allows us to configure database mapping inside DbContext."

Why Do We Need Fluent API?

Data Annotations are good for simple configurations.

Examples:

[Key]

[Required]

[StringLength]

But some configurations are complex.

Examples:

- ✓ Composite Primary Key
- ✓ One-to-One Relationship
- ✓ Cascade Delete
- ✓ Column Precision
- ✓ Table Mapping
- ✓ Default Values

These are easier with Fluent API.

Data Annotation vs Fluent API

Data Annotation

Configuration inside Model Class

Example:

[Required]

```
public string Title { get; set; }
```

Fluent API

Configuration inside DbContext

Example:

```
modelBuilder.Entity<Book>()  
    .Property(b => b.Title)  
    .IsRequired();
```

Where Do We Write Fluent API?

Inside:

OnModelCreating()

Method of DbContext.

Example

```
public class ApplicationDbContext  
    : DbContext  
{  
    protected override void
```

```
OnModelCreating(  
    modelBuilder modelBuilder)  
{  
    base.OnModelCreating(  
        modelBuilder);  
}  
}
```

BOOK MODEL

```
public class Book  
{  
    public int Id { get; set; }  
    public string Title  
    {  
        get;  
        set;  
    }  
    public decimal Price  
    {  
        get;  
        set;  
    }  
}
```

1. REQUIRED FIELD

Using Data Annotation

[Required]

```
public string Title  
{  
    get;  
    set;  
}
```

Using Fluent API

```
protected          override          void
OnModelCreating(
ModelBuilder modelBuilder)
{
modelBuilder.Entity<Book>()
.Property(x => x.Title)
.IsRequired();
} -----
```

- 2. MAX LENGTH

```
protected          override          void
OnModelCreating(
ModelBuilder modelBuilder)
{
modelBuilder.Entity<Book>()
.Property(x          =>          x.Title)
.HasMaxLength(100);
} -----
```

- Database

nvarchar(100) -----

----- 3. TABLE NAME

```
protected          override          void
OnModelCreating(
ModelBuilder modelBuilder)
{
modelBuilder.Entity<Book>()
.ToTable("Books");
} -----
```

-

Database Table

Books

4. COLUMN NAME

protected override void

OnModelCreating(

ModelBuilder modelBuilder)

{

modelBuilder.Entity<Book>()

.Property(x => x.Title)

.HasColumnName(

"Book_Title");

}

Database Column

Book_Title

5. PRIMARY KEY

protected override void

OnModelCreating(

ModelBuilder modelBuilder)

{

modelBuilder.Entity<Book>()

.HasKey(x => x.Id);

}

Primary Key

Id

6. COMPOSITE PRIMARY KEY

public class OrderDetail

{

```
public int OrderId
{
    get;
    set;
}
public int ProductId
{
    get;
    set;
}
}
```

Fluent API

```
modelBuilder.Entity<OrderDetail>()
    .HasKey(x =>
        new
        {
            x.OrderId,
            x.ProductId
        });
```

Data Annotations Cannot Easily Do This.

7. DECIMAL PRECISION

```
public decimal Price
{
    get;
    set;
}
```

Fluent API

```
modelBuilder.Entity<Book>()
    .Property(x => x.Price)
```

.HasPrecision(18,2); -----

----- Database

decimal(18,2) -----

----- 8. DEFAULT VALUE

modelBuilder.Entity<Book>()

.Property(x => x.Price)

.HasDefaultValue(100); -----

----- Default Price

100 -----

----- 9. IGNORE PROPERTY

modelBuilder.Entity<Book>()

.Ignore(x => x.TempValue); -----

----- Property

not created in database. -----

----- 10. ONE TO

MANY RELATIONSHIP Category

|

|

| One

|

V

Many Books -----

----- Models

public class Category

{

public int Id

```

{
get;
set;
}
public ICollection<Book> Books
{
get;
set;
}
}
public class Book
{
public int Id
{
get;
set;
}
public int CategoryId
{
get;
set;
}
public Category Category
{
get;
set;
}
}

```

Fluent API

```

modelBuilder.Entity<Book>()
    .HasOne(x => x.Category)
    .WithMany(x => x.Books)

```

```
.HasForeignKey(  
x => x.CategoryId); -----
```

----- 11. ONE TO ONE

RELATIONSHIP Student

|

|

| One

|

V

One Address -----

```
modelBuilder.Entity<Student>()
```

```
.HasOne(x => x.Address)
```

```
.WithOne(x      =>      x.Student)
```

```
.HasForeignKey<Address>(
```

```
x => x.StudentId); -----
```

----- 12. MANY TO MANY

RELATIONSHIP Student

|

|

| Many

|

V

Course -----

```
----- modelBuilder.Entity<Student>()
```

```
.HasMany(x      =>      x.Courses)
```

```
.WithMany(x => x.Students); -----
```

----- 13.

CASCADE DELETE

```
modelBuilder.Entity<Book>()  
.HasOne(x => x.Category)  
.WithMany(x => x.Books)  
.OnDelete(  
DeleteBehavior.Cascade);
```

Delete Category
Automatically deletes Books.

14. RESTRICT DELETE

```
.OnDelete(  
DeleteBehavior.Restrict);
```

Prevent delete if child records exist.

WHEN TO USE FLUENT API?

- ✓ Complex Relationships
 - ✓ Composite Keys
 - ✓ Column Precision
 - ✓ Default Values
 - ✓ Table Mapping
 - ✓ Enterprise Applications
-

WHEN TO USE DATA ANNOTATIONS?

- ✓ Simple Validation
 - ✓ Required Fields
 - ✓ Length Restrictions
 - ✓ Display Names
-

BEST PRACTICE

Use Both Together
Data Annotations
For Validation

Fluent API

For Database Configuration

INTERVIEW QUESTIONS

Q. What is Fluent API?

Fluent API is a way to configure entities, relationships, and database schema using code inside `OnModelCreating()`.

Q. Where is Fluent API written?

Inside `OnModelCreating()` method of `DbContext`.

Q. Which is more powerful: Data Annotations or Fluent API?

Fluent API

Q. Why use Fluent API?

To configure complex relationships, composite keys, constraints, and database behavior.

Q. Can Fluent API configure Primary Keys?

Yes.

`HasKey()`

Q. Can Fluent API configure Foreign Keys?

Yes.

`HasForeignKey()`

Q. What method is used for One-to-Many Relationship?

`HasOne()`

`WithMany()`

Q. What method is used for Composite Primary Key?

`HasKey()`

MEMORY TRICK

Data Annotation

Simple Configuration

Inside Model

Fluent API

Advanced Configuration

Inside DbContext

One-Line Interview Answer

Fluent API is EF Core's code-based configuration approach that uses the `OnModelCreating()` method to configure entities, keys, relationships, constraints, and database behavior with greater flexibility than Data Annotations.

=====

DATABASE MIGRATIONS IN ENTITY FRAMEWORK CORE (EF CORE)

What is a Migration?

Migration is a feature in EF Core that keeps the database schema synchronized with Model classes.

Simple Definition:

"Migration converts Model changes into Database changes."

Why Do We Need Migrations?

Suppose we have:

```
public class Book
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Title
```

```
    {
```

```
        get;
```

```
        set;
```

```
    }
```

```
}
```

Database Created Successfully.

Later Requirement Changes

Add Price Column

```
public class Book
{
    public int Id { get; set; }
    public string Title
    {
        get;
        set;
    }
    public decimal Price
    {
        get;
        set;
    }
}
```

Question

How will database know that a new column was added?

Answer:

Migration

Without Migration

Model Updated

Database Not Updated

Application Errors

With Migration

Model Updated

|

V

Migration Created

I

V

Database Updated

Migration Workflow

Model Change

I

V

Add Migration

I

V

Migration File Created

I

V

Update Database

I

V

Database Updated

Step 1: Create Model

public class Book

{

public int Id { get; set; }

public string Title

{

get;

set;

}

}

Step 2: Create DbContext

public class ApplicationDbContext

```
: DbContext
{
public DbSet<Book> Books
{
get;
set;
}
}
```

Step 3: Add Initial Migration

Package Manager Console

Add-Migration InitialCreate

What Happens?

EF Core compares:

Models

DbContext

Creates Migration File

Example

Migrations

|

+-- 20260618_InitialCreate.cs

Migration File Contains

CreateTable(

name: "Books",

columns: table => new

{

Id = table.Column<int>(),

Title =

table.Column<string>()

});

Step 4: Create Database

Update-Database

What Happens?

Migration Executes

Database Created

Books Table Created

Database Structure

Books

Id

Title

Adding New Column

Model Updated

public class Book

{

public int Id { get; set; }

public string Title

{

get;

set;

}

public decimal Price

{

get;

set;

}

}

Create New Migration

Add-Migration AddPriceColumn

Generated Migration

```
migrationBuilder.AddColumn<
decimal>(
name: "Price",
table: "Books");
```

Apply Migration

Update-Database

Database Updated

Books

Id

Title

Price

Migration Commands

1. Add Migration

Add-Migration MigrationName

Example

Add-Migration InitialCreate

2. Update Database

Update-Database

Applies pending migrations.

3. Remove Migration

Remove-Migration

Removes latest migration.

4. List Migrations

Get-Migration

Displays all migrations.

5. Rollback Database

Update-Database MigrationName

Example

Update-Database InitialCreate

Database returns to that migration.

Migration History Table

EF Core creates:

__EFMigrationsHistory

Purpose

Stores:

Applied Migrations

Migration Dates

Migration Versions

Example

__EFMigrationsHistory

InitialCreate

AddPriceColumn

AddCategoryTable

Real World Example

Version 1

Book Table

Id

Title

Version 2

Added Price

Version 3

Added Category

Version 4

Added Publisher

Each change tracked by migrations.

Benefits of Migrations

✓ Database Version Control

✓ Team Collaboration

✓ Easy Database Updates

✓ Automatic Schema Changes

✓ Safe Deployment

Code First Migration Flow

Create Model

|

V

Add-Migration

|

V

Migration File

|

V

Update-Database

|

V

Database Created/Updated

Common Migration Naming

InitialCreate

AddBookTable

AddPriceColumn

AddCategoryTable

UpdateBookSchema

Best Practices

✓ One Migration Per Change

✓ Use Meaningful Names

✓ Review Migration File

✓ Commit Migrations To Git

✓ Never Edit Migration Randomly

Migration Example

Version 1

```
public class Book
```

```
{
```

```
    public int Id { get; set; }
```

```
    public string Title
```

```
{
```

```
        get;
```

```
        set;
```

```
    }
```

```
}
```

Migration:

InitialCreate

Version 2

Add Price

Migration:

AddPriceColumn

Version 3

Add Category

Migration:

AddCategoryTable

INTERVIEW QUESTIONS

Q. What is Migration?

Migration is a feature that keeps the database schema synchronized with Model classes. --

Q. Why do we need Migrations?

To apply model changes to the database without manually writing SQL scripts. -----

Q. Which command creates a migration?

Add-Migration MigrationName

Q. Which command applies migration to database?

Update-Database

Q. Which command removes latest migration?

Remove-Migration

Q. What is __EFMigrationsHistory table?

A table used by EF Core to track applied migrations. -----

Q. What is Code First Migration?

A process where database schema is generated and updated from C# model classes. -----

Q. Can we rollback migrations?

Yes.

Using:

Update-Database MigrationName

MEMORY TRICK

Model Changed

|

Add-Migration

|

Migration File Created

|

Update-Database

|

Database Updated

One-Line Interview Answer

Database Migration is an EF Core feature that tracks model changes and automatically generates database schema updates, allowing developers to evolve the database without manually writing SQL scripts. CRUD OPERATIONS USING ENTITY FRAMEWORK CORE (EF CORE)

CRUD =

C → Create

R → Read

U → Update

D → Delete

What is CRUD?

CRUD operations are the most common database operations performed in applications.

Examples:

Create Book

Read Books

Update Book

Delete Book

Book Model

```
public class Book
{
    public int Id { get; set; }
    public string Title
    {
        get;
        set;
    }
    public string Author
    {
        get;
        set;
    }
    public decimal Price
    {
        get;
        set;
    }
}
```

DbContext

```
public class ApplicationDbContext
: DbContext
{
    public DbSet<Book> Books
    {
        get;
        set;
    }
}
```

1. CREATE OPERATION

Purpose:

Insert new record into database.

Code

```
var book = new Book
{
    Title = "Clean Code",
    Author = "Robert Martin",
    Price = 999
};
_context.Books.Add(book);
_context.SaveChanges();
```

SQL Generated

INSERT INTO Books

```
(
    Title,
    Author,
    Price
)
VALUES
(
    'Clean Code',
    'Robert Martin',
    999
)
```

Add Multiple Records

```
var books = new List<Book>()
{
    new Book
    {
        Title="Book1"
```

```
},  
new Book  
{  
    Title="Book2"  
}  
};  
_context.Books.AddRange(books);  
_context.SaveChanges();  
-----
```

2. READ OPERATION

Purpose:

Retrieve records from database.

Get All Books

```
var books =  
_context.Books.ToList();  
-----
```

SQL Generated

```
SELECT *  
FROM Books  
-----
```

Get Book By Id

```
var book =  
_context.Books.Find(1);  
-----
```

SQL Generated

```
SELECT *  
FROM Books  
WHERE Id = 1  
-----
```

Using FirstOrDefault

```
var book =  
_context.Books  
-----
```

```
.FirstOrDefault(
```

```
x => x.Id == 1);
```

Search Books

```
var books =
```

```
_context.Books
```

```
.Where(
```

```
x => x.Price > 500)
```

```
.ToList();
```

SQL Generated

```
SELECT *
```

```
FROM Books
```

```
WHERE Price > 500
```

Sorting

```
var books =
```

```
_context.Books
```

```
.OrderBy(
```

```
x => x.Title)
```

```
.ToList();
```

Descending Order

```
var books =
```

```
_context.Books
```

```
.OrderByDescending(
```

```
x => x.Price)
```

```
.ToList();
```

3. UPDATE OPERATION

Purpose:

Modify existing record.

Step 1

Find Record

var book =

_context.Books.Find(1);

Step 2

Update Values

book.Price = 1200;

book.Title = "Updated Book";

Step 3

Save Changes

_context.SaveChanges();

SQL Generated

UPDATE Books

SET

Price = 1200,

Title = 'Updated Book'

WHERE

Id = 1

Alternative Update

_context.Books.Update(book);

_context.SaveChanges();

4. DELETE OPERATION

Purpose:

Remove record from database.

Step 1

Find Record

```
var book =  
_context.Books.Find(1); -----
```

----- Step 2

Delete

```
_context.Books.Remove(book); -----
```

----- Step 3

Save Changes

```
_context.SaveChanges(); -----
```

----- SQL

Generated

DELETE FROM Books

WHERE Id = 1 -----

----- Delete Multiple Records

```
_context.Books.RemoveRange(books);
```

```
_context.SaveChanges(); -----
```

----- COMPLETE

CRUD EXAMPLE CREATE

```
var book = new Book();
```

```
_context.Books.Add(book);
```

```
_context.SaveChanges(); -----
```

----- READ

```
var books =
```

```
_context.Books.ToList(); -----
```

----- UPDATE

```
var book =
```

```
_context.Books.Find(1);
```

```
book.Price = 1000;
_context.SaveChanges();
```

DELETE

```
var book =
_context.Books.Find(1);
_context.Books.Remove(book);
_context.SaveChanges();
```

CRUD IN REPOSITORY PATTERN

Interface

```
public interface IBookRepository
{
    List<Book> GetAll();
    Book GetById(int id);
    void Add(Book book);
    void Update(Book book);
    void Delete(int id);
}
```

Repository

```
public class BookRepository
: IBookRepository
{
    private readonly
    ApplicationDbContext _context;
    public BookRepository(
    ApplicationDbContext context)
    {
        _context = context;
    }
    public List<Book> GetAll()
    {
```

```

return _context.Books.ToList();
}

public Book GetById(int id)
{
return _context.Books.Find(id);
}

public void Add(Book book)
{
_context.Books.Add(book);
_context.SaveChanges();
}

public void Update(Book book)
{
_context.Books.Update(book);
_context.SaveChanges();
}

public void Delete(int id)
{
var book =
_context.Books.Find(id);
_context.Books.Remove(book);
_context.SaveChanges();
}
}

```

ASYNC CRUD OPERATIONS CREATE

```

await _context.Books.AddAsync(book);
await _context.SaveChangesAsync(); --

```

READ

```

var books =

```

```
await _context.Books  
.ToListAsync(); -----
```

----- READ BY ID

```
var book =
```

```
await _context.Books  
.FindAsync(1); -----
```

----- UPDATE

```
_context.Books.Update(book);  
await _context.SaveChangesAsync();  
-----
```

DELETE

```
_context.Books.Remove(book);  
await _context.SaveChangesAsync();  
-----
```

MOST COMMON EF METHODS

Add()

AddRange()

Find()

FindAsync()

ToList()

ToListAsync()

Where()

OrderBy()

Update()

Remove()

RemoveRange()

SaveChanges()

```
SaveChangesAsync() -----
```

----- CRUD FLOW

Controller

I

V

Repository

I

V

DbContext

I

V

Database

INTERVIEW QUESTIONS

Q. What is CRUD?

Create, Read, Update, Delete operations.

Q. Which method inserts a record?

Add()

Q. Which method retrieves all records?

ToList()

Q. Which method retrieves a record by Primary Key?

Find()

Q. Which method updates a record?

Update()

Q. Which method deletes a record?

Remove()

Q. Why is SaveChanges() required?

To commit changes to the database.

Q. Difference between Find() and FirstOrDefault()?

Find()

Searches using Primary Key.

FirstOrDefault()

Can search using any condition.

Q. Which methods are used for Async CRUD?

AddAsync()

FindAsync()

ToListAsync()

SaveChangesAsync()

MEMORY TRICK

CREATE → Add()

READ → Find(), ToList()

UPDATE → Update()

DELETE → Remove()

COMMIT → SaveChanges()

One-Line Interview Answer

CRUD operations in EF Core are performed using DbSet methods such as Add(), Find(), ToList(), Update(), Remove(), and SaveChanges(), allowing developers to create, read, update, and delete database records using C# objects instead of SQL.

ASP.NET Core MVC (.NET 8) BookStore Trainer Handbook

Sprint 1 – MVC Fundamentals

Created By: Sumit Chaturvedi Founder & Corporate Trainer RightShift Solutions

Sprint Goal

By the end of this sprint, students will:

Understand ASP.NET Core MVC architecture

Learn the responsibilities of Model, View, and Controller

Understand how a browser request travels through an MVC application

Create their first ASP.NET Core MVC application

Build the first BookStore page

Understand project structure

Create their first Controller and View

Learn MVC interview fundamentals

Business Problem

Imagine a local bookstore named Knowledge Hub Books.

The owner wants a website where customers can:

View books

Search books

Read book details

Purchase books

The owner also wants employees to:

Add books

Update books

Remove books

Initially, a developer creates everything inside a single file:

HTML

Business Logic

Database Code

After a few months:

Code becomes difficult to maintain

Bugs increase

New features take longer

Multiple developers cannot work efficiently

The bookstore needs a structured architecture.

This is where MVC comes in.

What is MVC?

MVC stands for:

M → Model

V → View

C → Controller

MVC is an architectural pattern used to separate application responsibilities.

Instead of writing everything in one place, MVC divides the application into three parts.

Why MVC Was Introduced

Before MVC, developers often created applications where:

HTML was mixed with C#

Business rules were mixed with UI

Database code was mixed with presentation code

Example:

```
// Bad Practice string sql = "SELECT * FROM Books"; var books = ExecuteQuery(sql);  
Response.Write("<table>"); foreach(var book in books) { Response.Write(book.Title); }  
Response.Write("</table>");
```

Problems:

Difficult to test

Difficult to maintain

Difficult to debug

Difficult to scale

MVC solves these problems.

Understanding MVC Through a Restaurant Example

Imagine you visit a restaurant.

Customer

Requests food.

Equivalent to:

Browser

Waiter

Takes your order.

Equivalent to:

Controller

Kitchen

Prepares food.

Equivalent to:

Model and Business Logic

Food Served

Displayed to customer.

Equivalent to:

View

MVC Architecture Diagram

Browser | v Controller | v Model | v Controller | v View | v Browser

Understanding Model

A Model represents data.

Example:

Book information.

```
public class Book { public int Id { get; set; } public string Title { get; set; } public string Author { get; set; } public decimal Price { get; set; } }
```

The Model contains data only.

Responsibilities of Model

Store data

Validate data

Transfer data between layers

Examples:

Book

Employee

Product

Customer

Order

Understanding View

A View is responsible for displaying information.

Example:

Book List Page

<h1>Books</h1>

View does not contain business logic.

View only displays data.

Responsibilities of View

Display information

Accept user input

Show validation messages

Examples:

Login Page

Registration Page

Product Page

Book Details Page

Understanding Controller

Controller acts as a middleman.

Responsibilities:

Receive requests

Call business logic

Fetch data

Return View

Example:

```
public class BooksController : Controller { public IActionResult Index() { return View(); } }
```

Responsibilities of Controller

Handle HTTP requests

Coordinate Model and View

Return responses

Examples:

Index() Details() Create() Edit() Delete()

MVC Request Lifecycle

Understanding request flow is critical.

Example URL:

/Books/Index

Step 1:

Browser sends request.

Step 2:

Routing identifies Controller.

BooksController

Step 3:

Action Method executes.

Index()

Step 4:

Controller fetches data.

Step 5:

Controller returns View.

Step 6:

View generates HTML.

Step 7:

HTML sent back to browser.

ASP.NET Core MVC Project Structure

BookStore | --- Controllers | --- Models | --- Views | --- wwwroot | --- Program.cs | --- appsettings.json

Controllers Folder

Contains all Controllers.

Example:

BooksController.cs HomeController.cs

Models Folder

Contains Models.

Example:

Book.cs Category.cs

Views Folder

Contains UI pages.

Example:

Views | --- Books | --- Index.cshtml

wwwroot Folder

Contains static files.

Examples:

CSS JavaScript Images Bootstrap

Program.cs

Application starting point.

```
var builder = WebApplication.CreateBuilder(args); builder.Services.AddControllersWithViews(); var  
app = builder.Build(); app.MapControllerRoute( name: "default", pattern:  
"{controller=Home}/{action=Index}/{id?}"); app.Run();
```

Why AddControllersWithViews()?

Registers MVC services.

Without this:

MVC won't work.

Creating Our First Model

Book.cs

```
public class Book { public int Id { get; set; } public string Title { get; set; } public string Author { get;  
set; } }
```

Creating Our First Controller

BooksController.cs

```
public class BooksController : Controller { public IActionResult Index() { return View(); } }
```

Creating Our First View

Index.cshtml

```
<h1>Welcome to BookStore</h1>
```

Running the Application

URL:

<https://localhost:5001/books/index>

Output:

Welcome to BookStore

Why Companies Use MVC

MVC is widely used because:

Maintainable

Testable

Scalable

Reusable

Real-World Industries Using MVC

Banking

Internet Banking

Insurance

Policy Management

Healthcare

Patient Management

ERP

Business Applications

Government

Citizen Portals

Industry Best Practices

Keep Controllers Thin

Bad:

Controller | Business Logic | Database Logic

Good:

Controller | Service | Repository

Use Strongly Typed Models

Avoid:

ViewBag

Prefer:

Model

Follow Separation of Concerns

Never mix:

HTML

Business Logic

Data Access

Common Mistakes

Mistake 1

Writing SQL inside Controller.

Mistake 2

Writing Business Logic in View.

Mistake 3

Creating Fat Controllers.

Mistake 4

Not Understanding Request Flow.

Mistake 5

Keeping Everything in One Class.

Interview Questions and Answers

Q1. What is MVC?

MVC is an architectural pattern that separates an application into Model, View, and Controller to improve maintainability and scalability.

Q2. What is the role of Model?

Model represents application data and business rules.

Q3. What is the role of View?

View displays information to users.

Q4. What is the role of Controller?

Controller receives requests, processes them, and returns responses.

Q5. Why MVC is better than putting all code in one file?

Because MVC provides:

Separation of concerns

Better testing

Better maintenance

Better scalability

Q6. What is IActionResult?

A common return type used by MVC action methods.

Example:

```
public IActionResult Index() { return View(); }
```

Q7. What is the entry point of ASP.NET Core MVC?

Program.cs

Q8. What is AddControllersWithViews()?

Registers MVC services into Dependency Injection Container.

Q9. What is the difference between Model and ViewModel?

Model represents actual data.

ViewModel is designed specifically for UI.

Q10. Explain MVC using a real-world example.

Restaurant Example:

Customer → Browser

Waiter → Controller

Kitchen → Model

Food → View

Hands-On Exercise

Create:

Model

Book

Properties:

Id Title Author

Controller

BooksController

Action:

Index()

View

Index.cshtml

Display:

Welcome to BookStore

Assignment

Build a simple MVC application:

Requirements:

Create Product Model

Create ProductController

Create Product View

Display 5 hardcoded products

Sprint Summary

In this sprint students learned: ✓

MVC Architecture

✓ Model

✓ View

✓ Controller

✓ Request Lifecycle

✓ Project Structure

✓ Program.cs

✓ First Controller

- ✓ First View
- ✓ First Model
- ✓ Industry Best Practices
- ✓ Interview Questions
- ✓ Hands-On Exercises

This sprint establishes the foundation for all future ASP.NET Core MVC development. Every feature implemented in upcoming sprints will rely on the concepts learned here.

Sprint 2 – Book Catalog

Sprint Goal

Build a Book Catalog page that displays all available books in the BookStore application.

Students will learn:

Collections in C#

List

Strongly Typed Views

Razor Syntax

foreach Loop

Passing Data from Controller to View

MVC Data Flow

Business Problem

A bookstore has hundreds of books.

Customers need a page where they can browse available books.

Without a catalog page:

Customers cannot view inventory

Sales decrease

Navigation becomes difficult

The first feature of almost every business application is displaying records.

Examples:

Employees

Products

Customers

Books

Orders

Concept Explanation

What is a Collection?

A collection stores multiple objects.

Example:

Book 1 Book 2 Book 3 Book 4

Instead of creating:

Book book1; Book book2; Book book3;

We use:

List<Book>

What is List?

List is a generic collection.

```
List<Book> books = new List<Book>();
```

Benefits:

Dynamic Size

Easy Search

Easy Add/Remove

Strong Typing

Why Use List?

Imagine Amazon.

Millions of products.

We cannot create separate variables.

Collections solve this problem.

MVC Data Flow

Repository | v Controller | v View | v Browser

Strongly Typed Views

A strongly typed view knows exactly what data it receives.

Example:

```
@model List<Book>
```

Benefits:

IntelliSense

Compile-time checking

Better readability

ViewBag vs Model

ViewBag

```
ViewBag.Books = books;
```

Problems:

No compile-time checking

Runtime errors

Model

```
return View(books);
```

Recommended.

Creating Book Repository

```
public class BookRepository { public List<Book> GetAll() { return new List<Book>() { new Book { Id = 1, Title = "Clean Code", Author = "Robert Martin" }, new Book { Id = 2, Title = "Design Patterns", Author = "GoF" } }; }
```

Controller Implementation

```
public IActionResult Index() { var books = _repository.GetAll(); return View(books); }
```

View Implementation

```
@model List<Book> <h2>Books</h2> <table> @foreach(var book in Model) { <tr>  
<td>@book.Title</td> <td>@book.Author</td> </tr> } </table>
```

Industry Notes

List pages are the most visited pages.

Examples:

Amazon Products

Flipkart Catalog

Employee Lists

Patient Lists

Interview Questions & Answers

What is List?

A generic collection used to store multiple objects.

What is a Strongly Typed View?

A view that receives a specific model type.

Why use Model instead of ViewBag?

Model provides type safety and IntelliSense.

What is Razor Syntax?

A syntax used inside .cshtml pages to mix HTML and C#.

Common Mistakes

Forgetting @model

Passing wrong model type

Using ViewBag unnecessarily

Null Model errors

Hands-On Exercise

Display:

5 Books

Inside a Bootstrap table.

Sprint Summary

Students learned:

✓ Collections

✓ List

✓ Strongly Typed Views

✓ Razor Syntax

✓ MVC Data Flow

✓ Catalog Page Development

Sprint 3 – Routing and Model Binding

Sprint Goal

Display details of a selected book and understand how URLs map to MVC actions.

Business Problem

Users need detailed information about a specific book.

Example:

Book List

User clicks:

Clean Code

System should display:

Title

Author

Price

Description

What is Routing?

Routing maps URLs to Controller Actions.

Example:

/Books/Details/5

MVC automatically maps:

Controller = Books Action = Details Id = 5

Route Parameters

Example:

/Books/Details/5

MVC extracts:

5

And passes it to:

Details(int id)

What is Model Binding?

Model Binding automatically converts request data into C# objects.

Example:

/Books/Details/5

Converted into:

int id = 5;

Automatically.

Why Model Binding is Important

Without Model Binding:

Request.Query["id"]

With Model Binding:

Details(int id)

Cleaner and safer.

Controller Implementation

```
public IActionResult Details(int id) { var book = _repository.GetByld(id); if(book == null) return  
NotFound(); return View(book); }
```

View Implementation

@model Book <h2>@Model.Title</h2> <p>@Model.Author</p>

When To Use Routing

Details Pages

Edit Pages

Delete Pages

Product Pages

Interview Questions & Answers

What is Routing?

Routing maps URLs to action methods.

What is Route Parameter?

A value inside the URL.

What is Model Binding?

Automatic conversion of request data into C# objects.

Why return NotFound()?

To return HTTP 404 when a record does not exist.

Common Mistakes

Wrong URL format

Ignoring null checks

Hardcoding URLs

Sprint Summary

✓ Routing

✓ Route Parameters

✓ Model Binding

✓ Details Page

✓ NotFound()

Sprint 4 – Create Book

Sprint Goal

Allow administrators to add new books.

Business Problem

A bookstore regularly receives new inventory.

Without Create functionality:

Inventory cannot be updated

Catalog becomes outdated

Understanding HTTP Methods

GET

Used to retrieve data.

Example:

/books/create

Displays form.

POST

Used to submit data.

Example:

Save Book

GET Action

```
public IActionResult Create() { return View(); }
```

POST Action

```
[HttpPost] public IActionResult Create(Book book) { _repository.Add(book); return  
RedirectToAction("Index"); }
```

What is PRG Pattern?

PRG means:

Post ↓ Redirect ↓ Get

Benefits:

Prevents duplicate submissions

Better user experience

Create View

```
<form method="post"> <input asp-for="Title" /> <input asp-for="Author" /> <button type="submit">  
Save </button> </form>
```

When To Use Create

Whenever users need to add:

Books

Products

Customers

Employees

Interview Questions & Answers

Difference Between GET and POST?

GET retrieves data.

POST submits data.

Why RedirectToAction?

Prevents duplicate form submission.

What is PRG Pattern?

Post → Redirect → Get.

Common Mistakes

Missing HttpPost

Returning View instead of Redirect

Not validating data

Sprint Summary

✓ Forms

✓ GET

✓ POST

✓ Create Operation

✓ PRG Pattern

Sprint 5 – Validation

Sprint Goal

Prevent invalid data from entering the system.

Business Problem

Users may enter:

Blank Title Negative Price Invalid Author

Bad data creates:

Incorrect Reports

Business Issues

Application Errors

What is Validation?

Validation ensures data correctness.

Data Annotation Attributes

Required

```
[Required] public string Title { get; set; }
```

StringLength

```
[StringLength(100)] public string Title { get; set; }
```

Range

```
[Range(1,10000)] public decimal Price { get; set; }
```

Server-Side Validation

Most important validation.

Never trust browser validation.

Always validate on server.

Model Example

```
public class Book { public int Id { get; set; } [Required] public string Title { get; set; } [Required]
public string Author { get; set; } [Range(1,10000)] public decimal Price { get; set; } }
```

ModelState

MVC stores validation results in:

ModelState

Example:

```
if(ModelState.IsValid) { // Save }
```

Controller Implementation

```
[HttpPost] public IActionResult Create(Book book) { if(!ModelState.IsValid) { return View(book); }  
_repository.Add(book); return RedirectToAction("Index"); }
```

Real World Example

Banking Applications validate:

Account Number

IFSC Code

Customer Name

Amount

Validation prevents fraud and errors.

Interview Questions & Answers

What is Validation?

Process of ensuring data correctness.

What is ModelState?

Object that stores validation results.

Why is Server-Side Validation Important?

Client-side validation can be bypassed.

What are Data Annotations?

Attributes used to define validation rules.

Common Mistakes

Trusting Client Validation

Ignoring ModelState

Missing Validation Attributes

Sprint Summary

✓ Validation

✓ Data Annotations

✓ ModelState

✓ Server Validation

✓ Error Prevention

Part 1 Completion Summary

Students can now:

✓ Display Books

- ✓ Navigate Between Pages
- ✓ View Book Details
- ✓ Create New Books
- ✓ Validate User Input
- ✓ Build Real MVC CRUD Foundations

Next Part: Sprint 6 – Edit Book Sprint 7 – Delete Book Sprint 8 – Layout & Bootstrap Sprint 9 – Partial Views

Part 2 – Intermediate MVC Development

Sprint 6 – Edit Book

Sprint Goal

Allow bookstore administrators to modify existing book information.

Students will learn:

Update Operations

Edit Forms

Hidden Fields

GET and POST Edit Actions

Record Identification using ID

CRUD Operations

Business Problem

Books rarely remain unchanged.

Examples:

Price Changes

Author Name Corrections

Description Updates

Stock Updates

Without Edit functionality:

Incorrect information remains visible

Customer trust decreases

Business processes suffer

Understanding CRUD

CRUD stands for:

Create Read Update Delete

Sprint 6 focuses on:

Update

Why Update is Important

Real-world data changes frequently.

Examples:

HR System

Employee Salary Updated

E-Commerce

Product Price Updated

Hospital

Patient Information Updated

BookStore

Book Price Updated

Edit Workflow

User Clicks Edit | v Load Existing Record | v Display Form | v User Changes Data | v Submit Form | v Save Changes

GET Edit Action

Displays existing data.

```
public IActionResult Edit(int id) { var book = _repository.GetById(id); if(book == null) return  
NotFound(); return View(book); }
```

POST Edit Action

Processes updated data.

```
[HttpPost] public IActionResult Edit(Book book) { _repository.Update(book); return  
RedirectToAction("Index"); }
```

Why Hidden Fields are Required

The record ID must travel back to the server.

```
<input type="hidden" asp-for="Id" />
```

Without ID:

Wrong record may update

Update may fail

Edit View

```
@model Book <form method="post"> <input asp-for="Id" type="hidden" /> <input asp-for="Title" />  
<input asp-for="Author" /> <input asp-for="Price" /> <button type="submit"> Save </button>  
</form>
```

Industry Notes

Most enterprise applications:

Track who modified data

Store modification timestamps

Maintain audit logs

Example:

Modified By Modified Date Modified Reason

Best Practices

✓ Validate data before update

✓ Check record existence

✓ Use audit logging

✓ Keep controllers thin

Interview Questions & Answers

What is Update Operation?

Modifying an existing record.

Why is ID important during Edit?

ID identifies which record should be updated.

Difference between Create and Edit?

Create adds a new record.

Edit modifies an existing record.

Why use Hidden Fields?

To preserve record identity.

Common Mistakes

■ Forgetting Hidden ID

■ Updating wrong record

■ Not validating user input

■ Ignoring null records

Hands-On Exercise

Update:

Title Author Price

for existing books.

Sprint Summary

✓ Update Operation

✓ Edit Forms

✓ Hidden Fields

✓ CRUD Concepts

✓ Enterprise Update Patterns

Sprint 7 – Delete Book

Sprint Goal

Allow administrators to remove books safely.

Business Problem

Not every book remains available forever.

Examples:

Out of Print

Removed Inventory

Incorrect Entry

Old Editions

Without Delete:

Catalog becomes cluttered.

Hard Delete vs Soft Delete

Hard Delete

Record permanently removed.

DELETE FROM Books

Advantages:

Saves storage

Disadvantages:

Data cannot be recovered

Soft Delete

Record marked inactive.

```
public bool IsDeleted { get; set; }
```

Example:

IsDeleted = true

Advantages:

Data Recovery

Auditing

Compliance

Preferred by enterprises.

Delete Workflow

User Clicks Delete | v Confirmation Screen | v Delete Action | v Redirect

GET Delete Action

```
public IActionResult Delete(int id) { var book = _repository.GetById(id); return View(book); }
```

POST Delete Action

```
[HttpPost] public IActionResult DeleteConfirmed(int id) { _repository.Delete(id); return  
RedirectToAction("Index"); }
```

Why Confirmation Screen?

Prevents accidental deletion.

Examples:

Amazon Admin Portal

Banking Applications

ERP Systems

Always ask:

Are you sure?

Industry Notes

Large organizations prefer:

✓ Soft Delete

✓ Audit Trails

✓ Approval Workflows

Interview Questions & Answers

What is Hard Delete?

Record permanently removed.

What is Soft Delete?

Record remains in database but marked inactive.

Which approach is preferred?

Soft Delete.

Why show confirmation screen?

Prevent accidental data loss.

Common Mistakes

■ Deleting without confirmation

■ No audit tracking

■ Using hard delete everywhere

Hands-On Exercise

Implement:

Delete Button Confirmation Page Delete Logic

Sprint Summary

✓ Delete Operations

✓ Hard Delete

✓ Soft Delete

✓ Confirmation Screens

✓ Data Protection

Sprint 8 – Layout and Bootstrap

Sprint Goal

Create a consistent and professional user interface.

Business Problem

Every page currently looks different.

Problems:

Repeated HTML

Difficult maintenance

Inconsistent UI

Need:

Common Header Common Footer Common Navigation

What is a Layout?

A Layout acts as a master template.

Example:

Header Navigation Content Footer

Shared across pages.

Layout Structure

```
<html> <body> <header> </header> <nav> </nav> @RenderBody() <footer> </footer> </body>  
</html>
```

What is RenderBody()?

Placeholder where page content appears.

Example:

```
@RenderBody()
```

Creating Layout

Views Shared _Layout.cshtml

Using Layout

```
@{ Layout = "_Layout"; }
```

What is Bootstrap?

Bootstrap is a UI framework.

Provides:

✓ Responsive Design

✓ Grid System

✓ Buttons

✓ Cards

✓ Navigation

Bootstrap Example

```
<button class="btn btn-primary"> Save </button>
```

Responsive Design

Website adapts to:

Desktop

Tablet

Mobile

Automatically.

Real World Example

Amazon

Flipkart

Netflix

Use reusable layouts.

Interview Questions & Answers

What is Layout?

Master page shared across multiple views.

What is `RenderBody()`?

Placeholder where page content renders.

Why use Bootstrap?

Rapid UI development.

Benefits of Layout?

Consistency and reusability.

Common Mistakes

■ Duplicate HTML

■ Multiple layouts unnecessarily

■ Ignoring responsive design

Hands-On Exercise

Create:

Header Footer Navigation Bootstrap Theme

Sprint Summary

✓ Layouts

✓ Shared Views

✓ Bootstrap

✓ Responsive Design

✓ Reusable UI

Sprint 9 – Partial Views

Sprint Goal

Create reusable UI components.

Business Problem

Many pages repeat:

Navigation Menu Login Panel Search Bar Footer Section

Copy-paste causes:

Maintenance issues

Code duplication

What is a Partial View?

A Partial View is a reusable UI component.

Example:

_SearchBar _LoginPanel _Menu

Why Use Partial Views?

Instead of:

Copy Paste Copy Paste

Use:

```
<partial name="_SearchBar" />
```

Creating Partial View

Views Shared _SearchBar.cshtml

Partial View Example

```
<form> <input placeholder="Search Books" /> </form>
```

Using Partial View

```
<partial name="_SearchBar" />
```

Partial View vs Layout

Layout

Entire Page Structure

Header Menu Footer

Partial View

Reusable Components

Search Bar Login Widget Menu

Real World Example

Amazon Search Bar

Google Login Widget

Netflix Navigation Menu

Benefits

✓ Reusability

✓ Easier Maintenance

✓ Cleaner Views

✓ Better Organization

Industry Notes

Large applications contain:

Hundreds of Views

Dozens of Partial Views

Used extensively in:

Banking

ERP

Healthcare

E-Commerce

Interview Questions & Answers

What is a Partial View?

Reusable UI component.

Difference between Layout and Partial View?

Layout = Page Template

Partial View = Reusable Component

Why use Partial Views?

Avoid duplication.

Where are Partial Views stored?

Usually:

Views/Shared

Common Mistakes

- Using Layout instead of Partial

- Duplicate UI code

- Large Partial Views

Hands-On Exercise

Create:

_SearchBar _CategoryMenu _BookCard

Use them throughout the application.

Sprint Summary

- ✓ Partial Views

- ✓ Reusable Components

- ✓ Clean UI Design

- ✓ Reduced Duplication

- ✓ Enterprise UI Patterns

Part 2 Completion Summary

Students can now:

- ✓ Edit Records

- ✓ Delete Records

- ✓ Design Professional UI

- ✓ Use Bootstrap

- ✓ Create Reusable Components

- ✓ Follow Enterprise MVC Practices

Ready for Part 3:

Sprint 10 – Search & Filtering

Sprint 11 – State Management

Sprint 12 – Authentication

Part 3 – Professional MVC Development

Sprint 10 – Search and Filtering

Sprint Goal

Implement powerful search and filtering capabilities in the BookStore application.

Students will learn:

Search Functionality

Query Strings

GET Forms

LINQ Filtering

Search Optimization

User Experience Best Practices

Business Problem

Imagine a bookstore with 10,000 books.

Without search:

Users waste time browsing

Customers leave the website

Sales decrease

Users want:

Search by Title Search by Author Search by Category

What is Search?

Search allows users to find information quickly.

Example:

Search: Clean Code

Returns:

Clean Code Clean Architecture

What is Filtering?

Filtering narrows results.

Example:

Category = Programming Price < 1000

Search Workflow

User Types Search | v Controller Receives Query | v Repository Filters Data | v Filtered Results Returned

Query String

Example URL:

/books?searchTerm=clean

MVC automatically binds:

```
public IActionResult Index(string searchTerm)
```

GET Form

Search should use GET.

```
<form method="get"> <input name="searchTerm" /> <button type="submit"> Search </button>
</form>
```

Controller Implementation

```
public IActionResult Index(string searchTerm) { var books = _repository.GetAll();
if(!string.IsNullOrEmpty(searchTerm)) { books = books .Where(x => x.Title.Contains(searchTerm))
.ToList(); } return View(books); }
```

Understanding LINQ Filtering

LINQ makes filtering easy.

```
books.Where(x => x.Title.Contains(searchTerm))
```

Meaning:

Return books whose title contains the entered text.

Multiple Filters

Example:

```
books.Where(x => x.Price < 1000 && x.Category == "Programming")
```

Real World Examples

Amazon:

Brand Price Rating Category

Flipkart:

Price Range Discount Availability

BookStore:

Author Title Category

Industry Notes

Search is one of the most used features.

Large applications use:

SQL Search

Elasticsearch

Azure Search

AI Search

Best Practices

- ✓ Use GET for Search
- ✓ Preserve Search Text
- ✓ Show Result Count
- ✓ Use Pagination

Interview Questions and Answers

Why use GET for Search?

Search does not modify data.

GET allows bookmarking URLs.

What is Query String?

Additional data passed in URL.

Example:

/books?searchTerm=clean

What is LINQ?

Language Integrated Query used to query collections.

Difference Between Search and Filter?

Search finds data.

Filter narrows existing data.

Common Mistakes

- Using POST for Search
- Ignoring Empty Search
- Case Sensitive Search
- No Result Handling

Hands-On Exercise

Implement:

Search by Title Search by Author Category Filter Sprint

Summary

- ✓ Search
- ✓ Filtering
- ✓ Query Strings
- ✓ LINQ
- ✓ User-Friendly Navigation

Sprint 11 – State Management

Sprint Goal

Understand how ASP.NET Core manages data across requests.

Students will learn:

Stateless HTTP

ViewData

ViewBag

TempData

Session

Cookies

Business Problem

User adds books to cart.

Moves to another page.

Cart disappears.

Why?

Because HTTP is stateless.

What is Stateless HTTP?

Each request is independent.

Example:

Request 1 No Memory Request 2 No Memory

Server forgets previous requests.

Why State Management Exists

To remember:

Logged-in User

Shopping Cart

User Preferences

Temporary Messages

ViewData

Stores data between Controller and View.

`ViewData["Title"] = "Book Store";`

View:

`@ViewData["Title"]`

ViewBag

Dynamic wrapper around ViewData.

```
ViewBag.Title = "Book Store";
```

View:

```
@ViewBag.Title
```

ViewData vs ViewBag

ViewData

```
ViewData["Name"]
```

Dictionary based.

ViewBag

```
ViewBag.Name
```

Dynamic object.

TempData

Used across redirects.

Controller:

```
TempData["Success"] = "Book Created";
```

Next Request:

```
@TempData["Success"]
```

Session

Stores user-specific data.

Program.cs

```
builder.Services.AddSession();
```

Middleware:

```
app.UseSession();
```

Saving Session Data

```
HttpContext.Session.SetString("UserName", "Sumit");
```

Reading Session Data

```
HttpContext.Session.GetString("UserName");
```

Cookies

Small data stored in browser.

Example:

```
Response.Cookies.Append("Theme", "Dark");
```

Real World Examples

Shopping Cart

Language Preference

Theme Selection

Remember Me

Industry Notes

Modern applications prefer:

Claims

JWT Tokens

Distributed Cache

instead of heavy Session usage.

Interview Questions and Answers

What is Stateless HTTP?

Server does not remember previous requests.

Difference between ViewData and ViewBag?

ViewData uses dictionary.

ViewBag uses dynamic properties.

What is TempData?

Stores data between requests.

What is Session?

Stores user-specific information.

Where are Cookies stored?

Browser.

Common Mistakes

■ Storing large data in Session

■ Overusing ViewBag

■ Sensitive Data in Cookies

Hands-On Exercise

Create:

Welcome Message Success Message Shopping Cart Count

using ViewBag, TempData and Session.

Sprint Summary

✓ ViewData

✓ ViewBag

✓ TempData

✓ Session

✓ Cookies

✓ Stateless HTTP

Sprint 12 – Authentication

Sprint Goal

Implement secure login functionality.

Students will learn:

Authentication

Authorization

Cookies

Claims

Login Flow

Business Problem

Not every user should access admin pages.

Example:

Admin Customer Guest

Need secure login.

Authentication vs Authorization

Authentication:

Who are you?

Authorization:

What can you do?

Real World Example

Airport Security

Authentication:

Passport Check

Authorization:

Access to Boarding Gate

Login Flow

User Enters Credentials | v Validate User | v Create Claims | v Create Cookie | v User Logged In

What are Claims?

Claims describe user information.

Examples:

Name Email Role

Creating Claims

```
var claims = new List<Claim>() { new Claim( ClaimTypes.Name, "Sumit"), new Claim( ClaimTypes.Role, "Admin") };
```

Creating Identity

```
var identity = new ClaimsIdentity( claims, "CookieAuth");
```

Sign In

```
await HttpContext .SignInAsync( new ClaimsPrincipal(identity));
```

Protecting Controller

```
[Authorize] public IActionResult Dashboard() { return View(); }
```

Role-Based Authorization

```
[Authorize(Roles="Admin")]
```

Only admins can access.

Logout

```
await HttpContext .SignOutAsync();
```

Real World Examples

Internet Banking

Hospital Systems

ERP Applications

Government Portals

Industry Notes

Most organizations use:

ASP.NET Core Identity

Azure AD

OAuth

OpenID Connect

Interview Questions and Answers

What is Authentication?

Verifying user identity.

What is Authorization?

Determining permissions.

Difference Between Authentication and Authorization?

Authentication identifies.

Authorization grants access.

What are Claims?

Pieces of user information.

What is Authorize Attribute?

Protects resources from unauthorized users.

What is Cookie Authentication?

Stores authentication information in encrypted cookies.

Common Mistakes

■ Storing Passwords in Plain Text

■ No Authorization

■ Weak Password Policies

■ Trusting Client Data

Hands-On Exercise

Implement:

Login Page Logout Page Admin Dashboard Role-Based Access

Sprint Summary

✓ Authentication

✓ Authorization

✓ Claims

✓ Cookies

✓ Login Flow

✓ Secure MVC Applications

Part 3 Completion Summary

Students can now:

✓ Search Books

✓ Filter Data

✓ Manage State

✓ Build Login Systems

✓ Implement Authorization

✓ Secure Business Applications

Ready for Part 4:

Sprint 13 – Security

Sprint 14 – Error Handling

Sprint 15 – Filters

Sprint 16 – Async Programming

ABOUT THIS BOOK

ASP.NET Core MVC is one of the most widely used frameworks for building modern, scalable, and secure web applications. This book is designed to help students, beginners, and developers understand ASP.NET Core MVC concepts in a simple and practical way.

From Dependency Injection and Repository Pattern to Razor Views, Routing, Validation, and MVC Architecture, every topic is explained with examples, interview questions, and real-world scenarios.

KEY FEATURES

-  Easy-to-understand explanations
-  Real-world examples
-  Interview-focused notes
-  MVC Architecture & Design Patterns
-  Dependency Injection & Repository Pattern
-  Validation, Routing & Razor Views
-  Beginner to Advanced Concepts



ASP.NET CORE MVC

WHO SHOULD READ THIS BOOK?

-  Students (BCA, MCA, B.Tech)
-  ASP.NET Beginners
-  .NET Developers
-  Interview Candidates
-  Self-Learners

ABOUT THE AUTHOR

Sumit Chaturvedi

ASP.NET Core Learner & Developer

Passionate about technology and teaching. I create simple, practical and industry-oriented content to help students and developers build strong concepts and develop real-world applications.

“ Learn Today, Build Tomorrow. ”



FIRST EDITION – 2026

PROFESSIONAL STUDY GUIDE

ASP.NET CORE MVC

